

>> IN THE NAME OF ALLAH, THE MOST GRACIOUS, THE MOST MERCIFUL <<

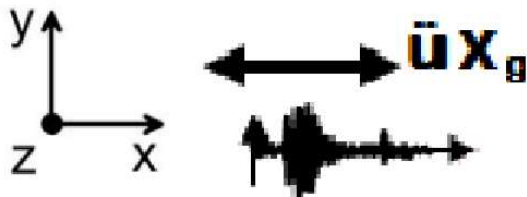
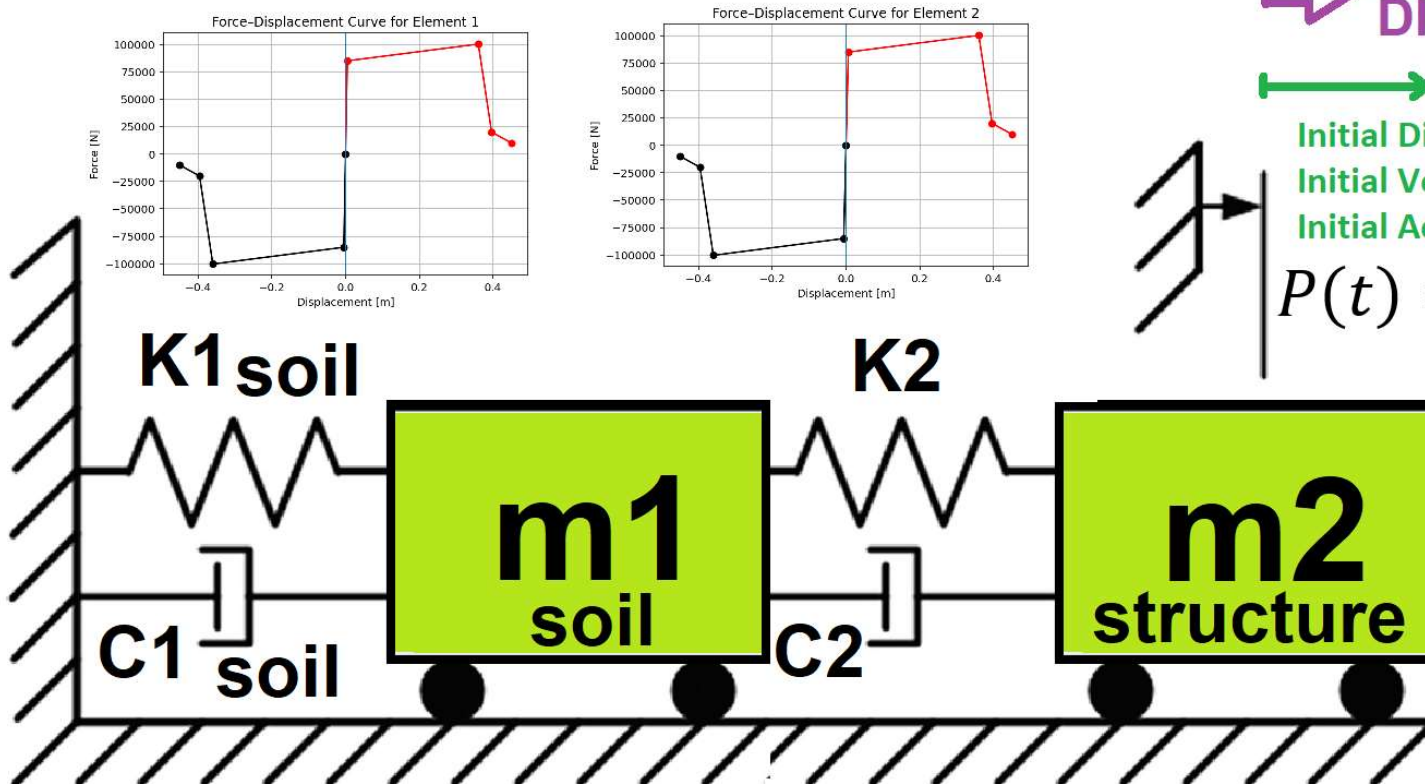
COMPREHENSIVE NONLINEAR SEISMIC ASSESSMENT OF A MULTI-DEGREE- FREEDOM SOIL STRUCTURE SYSTEM: AN OPENSEES FRAMEWORK FOR STATIC PUSHOVER, CYCLIC DEGRADATION, STATIC TIME-HISTORY AND DYNAMIC TIME- HISTORY ANALYSIS

THIS PROGRAM WRITTEN BY SALAR DELAVAR GHASHGHAEE (QASHQAI)

➡ INCREMENTAL
DISPLACEMENT

↔
Initial Displacement
Initial Velocity
Initial Acceleration

$$P(t) = P_0 e^{-0.05\bar{\omega}t} \sin(\bar{\omega}t)$$



$$\text{Structural Ductility Damage Index} = \frac{\Delta_d - \Delta_y}{\Delta_u - \Delta_y}$$

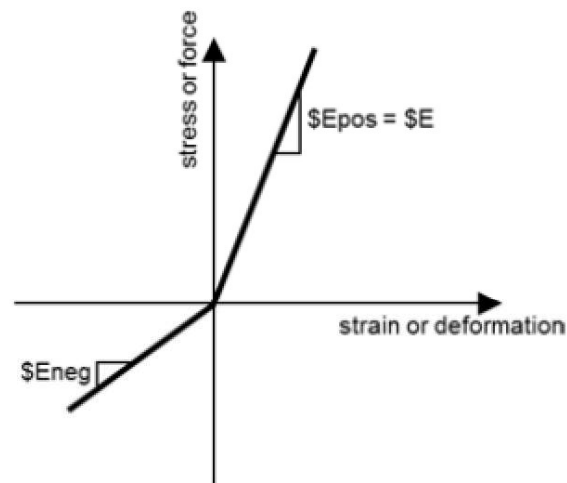
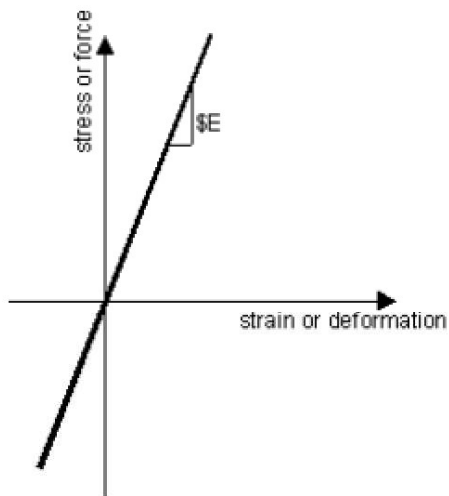
Δ_d = Lateral Displaement from Dynamic Analysis

Δ_y = Lateral Yield Displaement from Pushover Analysis

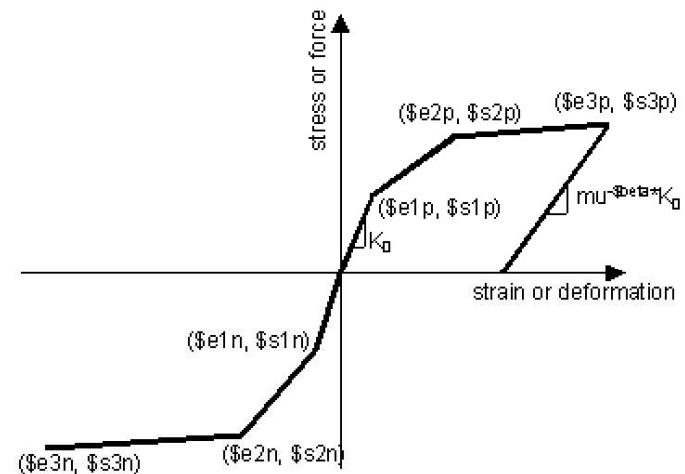
Δ_u = Lateral Ultimate Displaement from Pushover Analysis

$$P(t) = P_0 e^{-0.05\bar{\omega}t} \sin(\bar{\omega}t)$$

$$m\ddot{u} + c\dot{u} + ku = P(t)$$

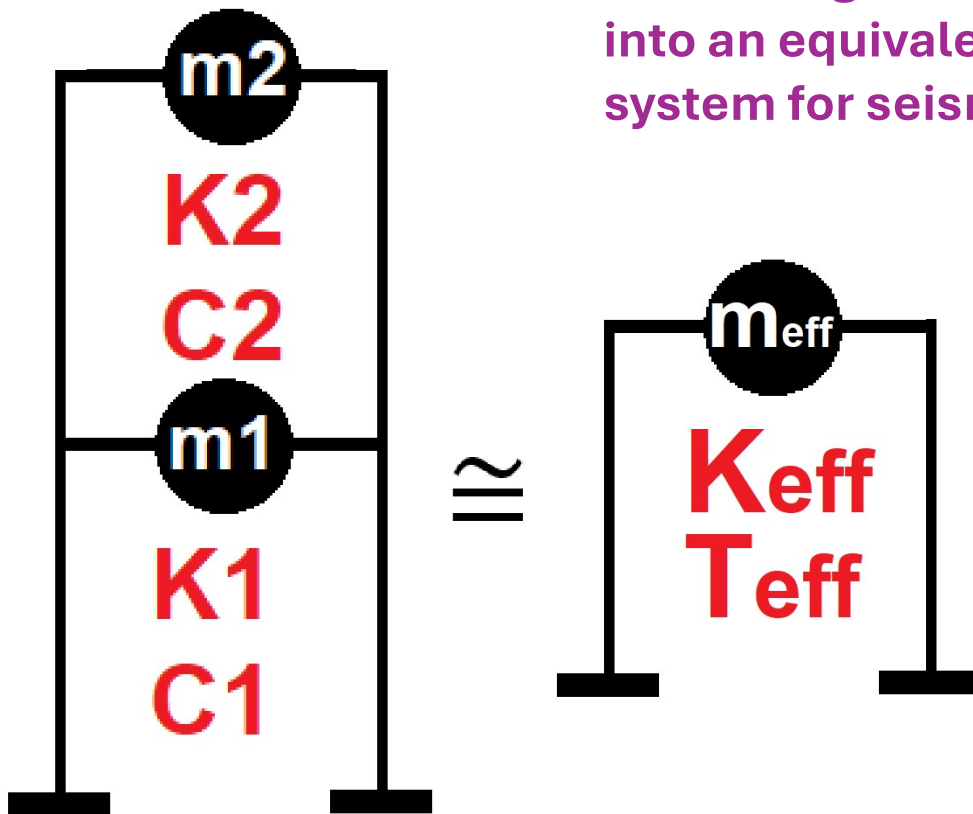


ELASTIC MATERIAL



INELASTIC MATERIAL

Displacement-based seismic analysis transformation, converting a multi-degree-of-freedom (MDOF) system into an equivalent single-degree-of-freedom (SDOF) system for seismic assessment



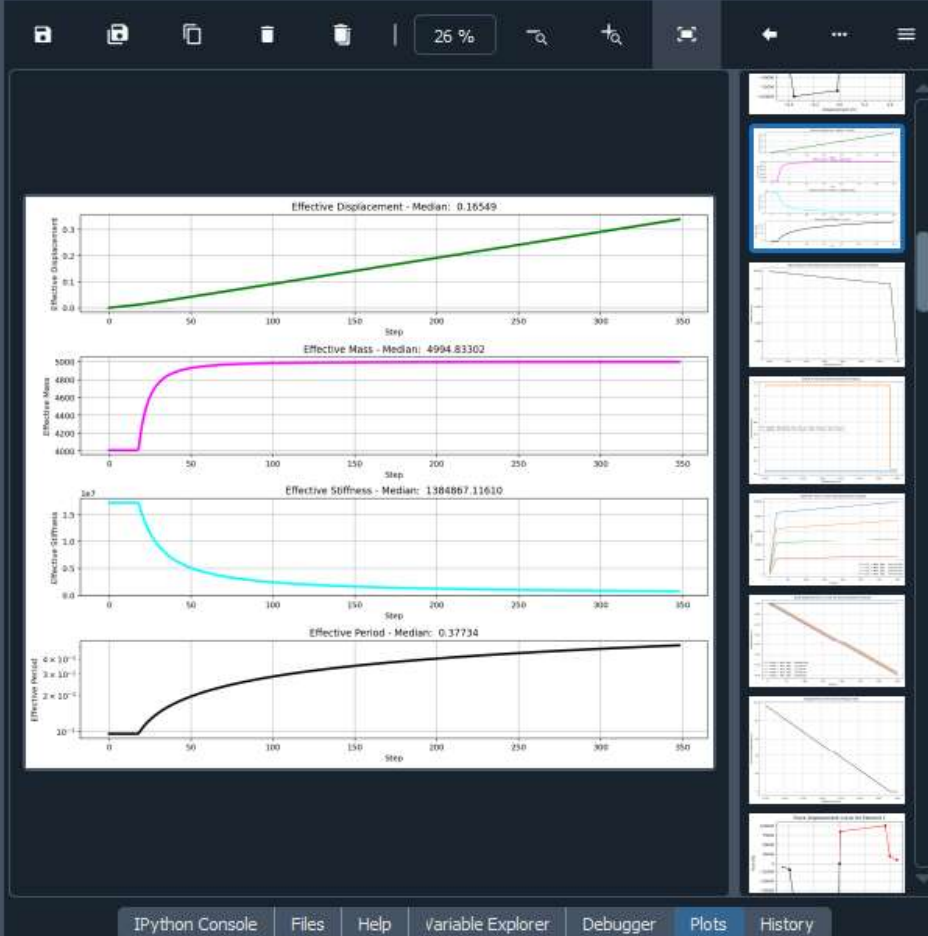
$$\Delta_d = \frac{\sum_{i=1}^n m_i \Delta_i^2}{\sum_{i=1}^n m_i \Delta_i} \quad m_{eff} = \frac{\sum_{i=1}^n m_i \Delta_i}{\Delta_d}$$

$$k_{eff} = \frac{\sum_{i=1}^n k_i \Delta_i}{\Delta_d} \quad T_{eff} = \frac{2\pi}{\sqrt{\frac{k_{eff}}{m_{eff}}}}$$

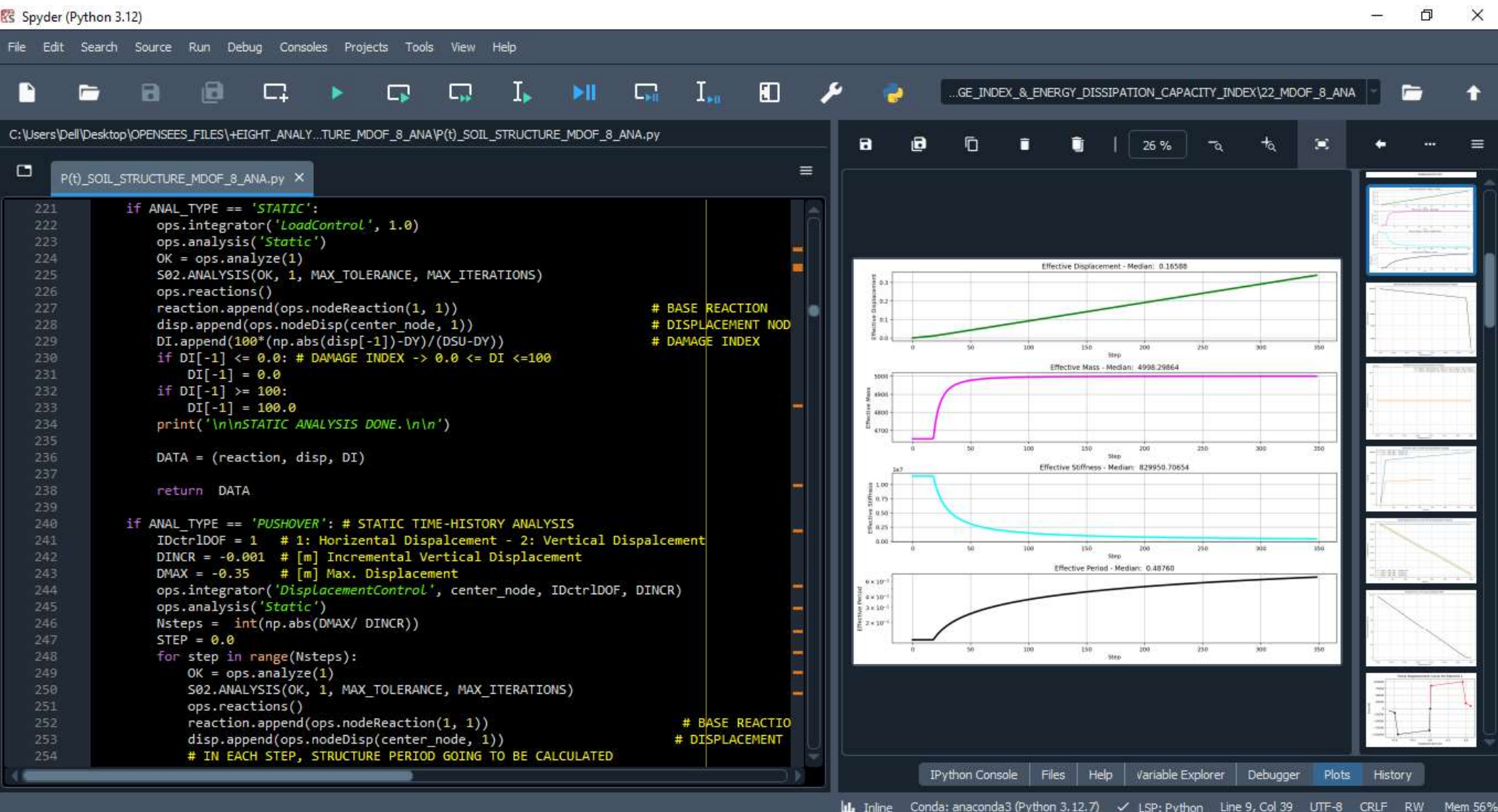
```

1 #####
2 # >> IN THE NAME OF ALLAH, THE MOST GRACIOUS, THE MOST MERCIFUL <<
3 # COMPREHENSIVE NONLINEAR SEISMIC ASSESSMENT OF A MULTI-DEGREE-FREEDOM SOIL STRUCTURE SYST
4 # FRAMEWORK FOR STATIC PUSHOVER, CYCLIC DEGRADATION, STATIC TIME-HISTORY AND DYNAMIC TIME-
5 #
6 #-----
7 # EQUIVALENT SDOF SYSTEM DERIVATION VIA DISPLACEMENT-BASED PUSHOVER ANAL
8 #-----
9 # P(t) = P0 sin(wt)
10 # P(t) = P0 exp(-0.05wt) sin(wt)
11 #-----
12 # ASSESSMENT OF DUCTILITY DAMAGE INDICES FOR STRUCTURAL ELEMENTS AND SYSTEMS AND E
13 # OF ENERGY DISSIPATION CAPACITY INDEX
14 #-----
15 # THIS PROGRAM WRITTEN BY SALAR DELAVAR GHASHGHAEI (QASHQAI)
16 # EMAIL: salar.d.ghashghaei@gmail.com
17 #####
18 Nonlinear Seismic Performance Assessment of a MDOF Soil Structure System:
19 An OpenSeesPy Framework for Material and Geometric Nonlinearity Under Static, Cyclic, and
20 
21 This OpenSeesPy script performs rigorous nonlinear static and dynamic analysis of a MDOF S
22 for performance-based earthquake engineering. The 2D model incorporates both material non
23 (elastic-perfectly plastic or hysteretic steel with strain hardening)
24 and geometric nonlinearity (corotational truss formulation) to capture P-delta effects
25 and large displacements-essential for collapse assessment.
26
27 Eight analysis protocols are implemented:
28 (1) [PERIOD] : Structural Period
29 (2) [STATIC] : Gravity load analysis establishing dead load state
30 (3) [PUSHOVER] : Displacement-controlled pushover generating full capacity curves
31 and plastic mechanism identification
32 (4) [CYCLIC_DISPLACEMENT] : Symmetric cyclic displacement protocol capturing hysteresis,
33 pinching behavior, and energy dissipation degradation
34 (5) [STATIC EXTERNAL TIME-DEPENDENT LOADING] : Static Analysis of External time-dependent

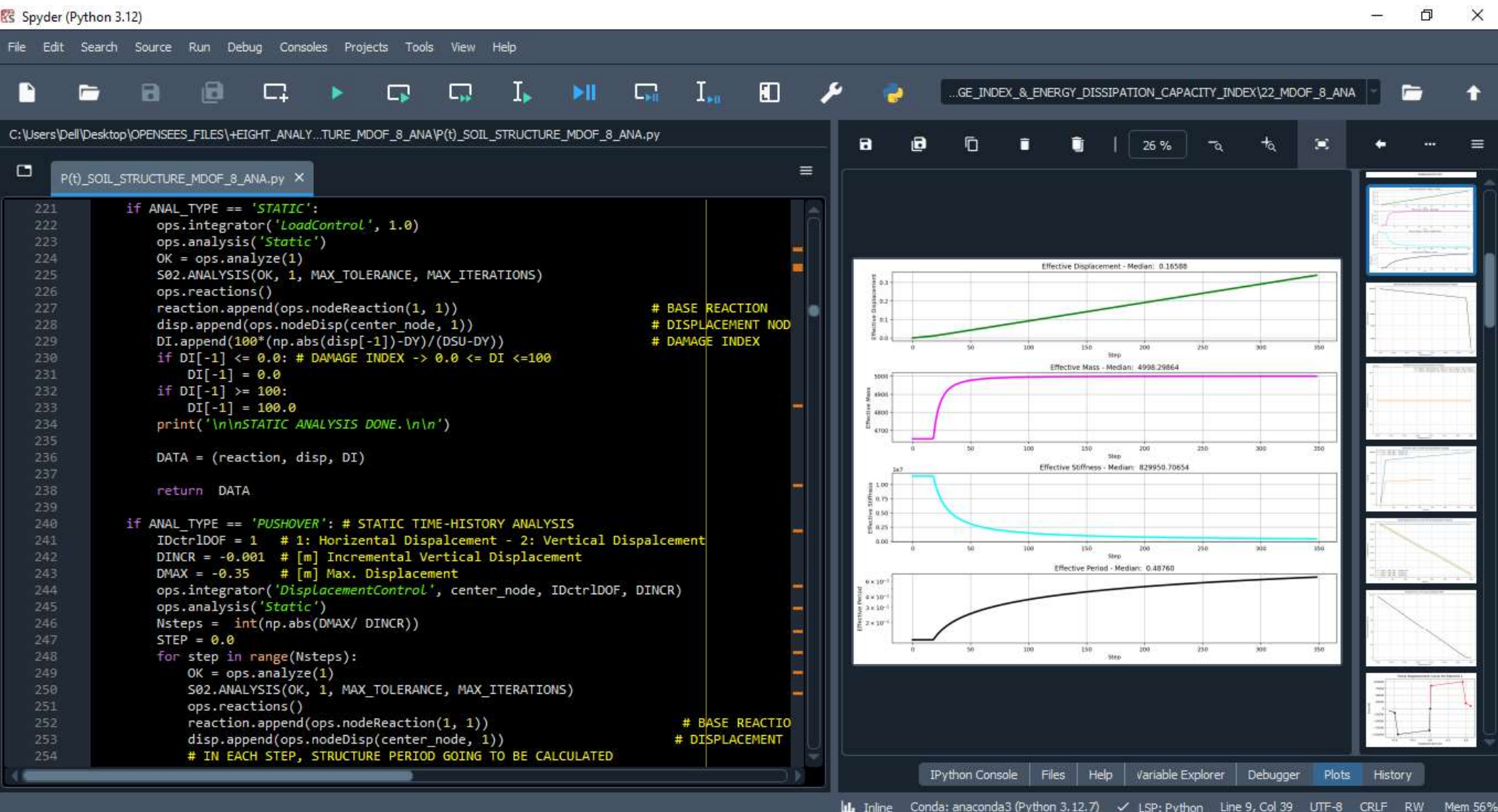
```

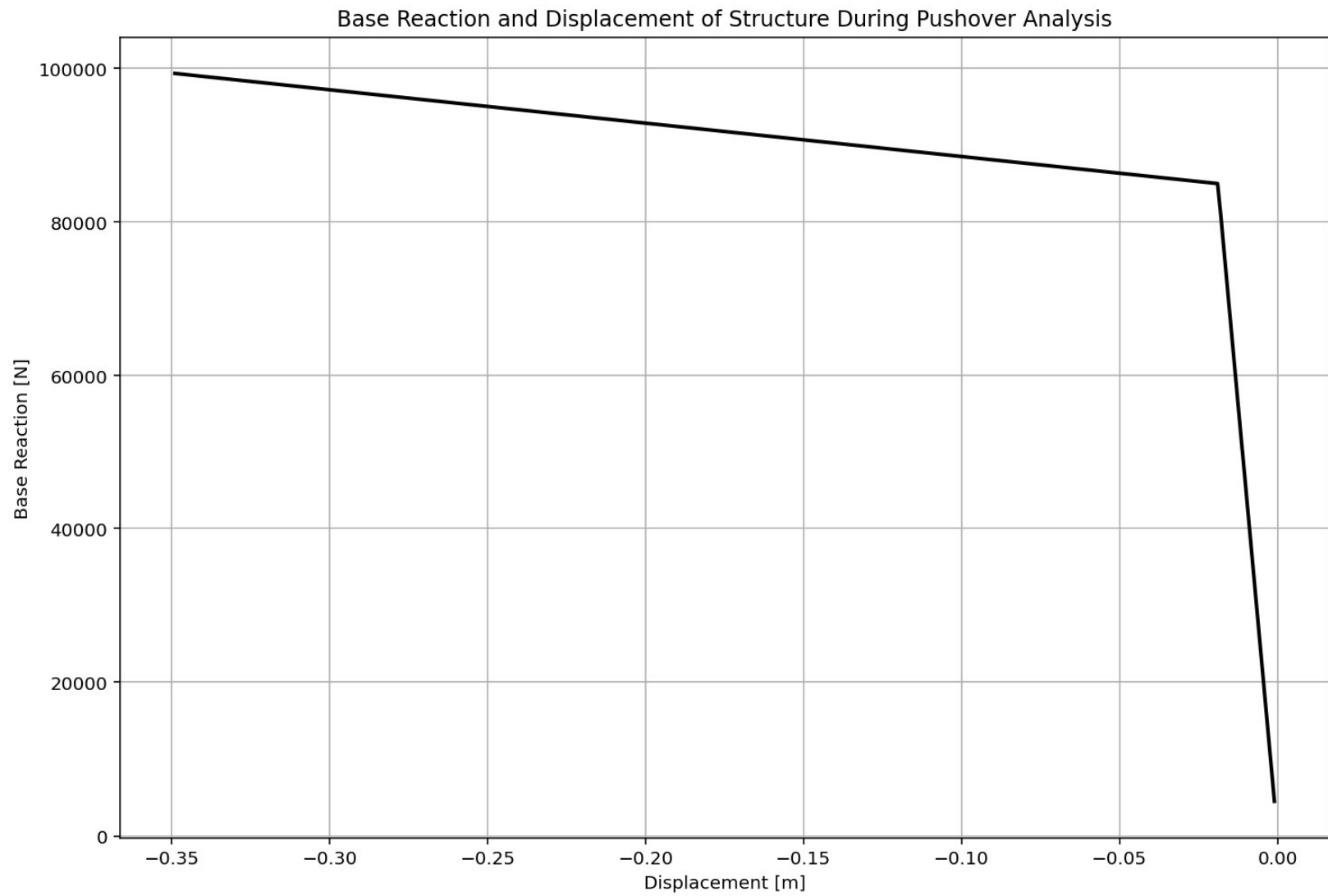


STATIC ANALYSIS

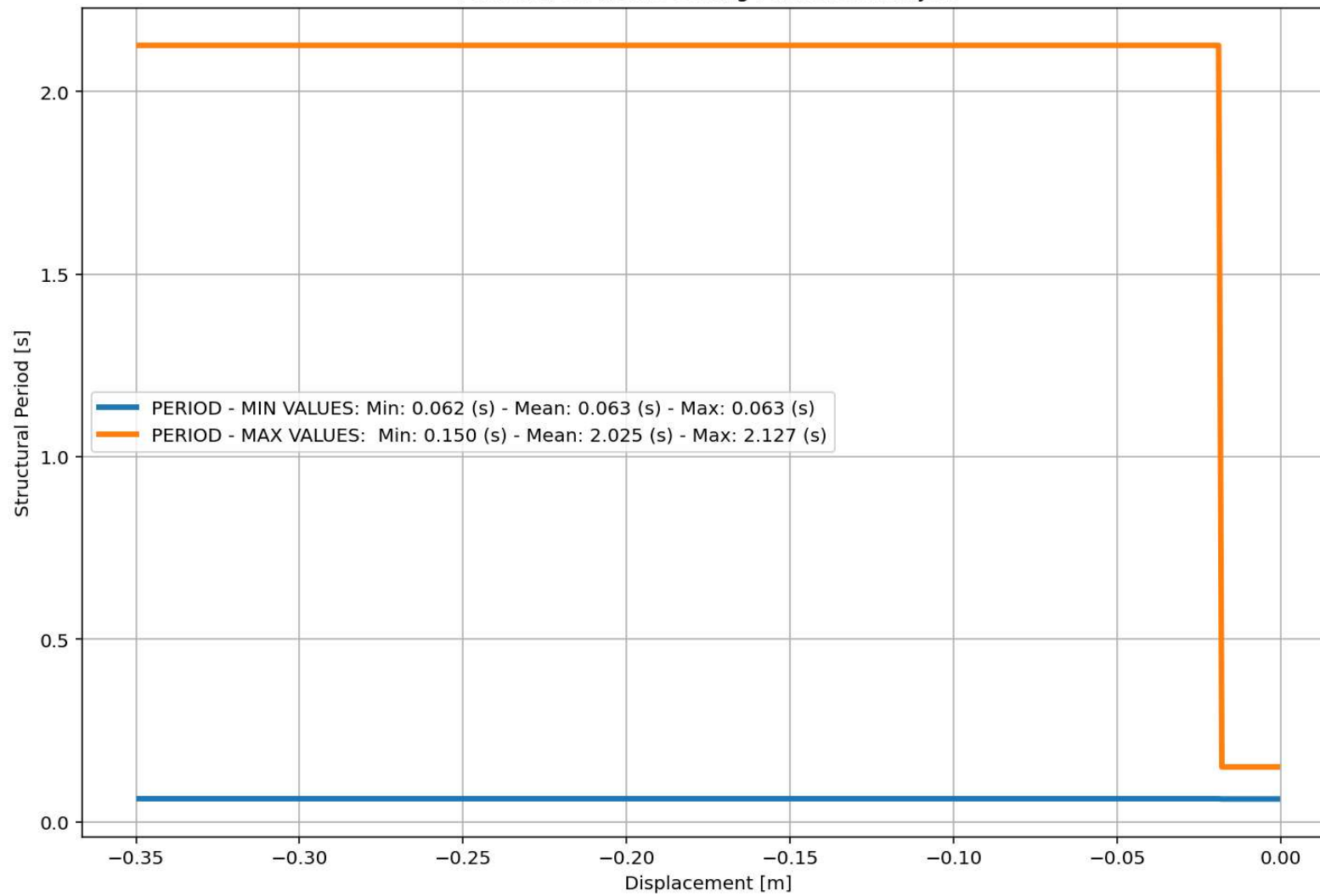


PUSHOVER ANALYSIS

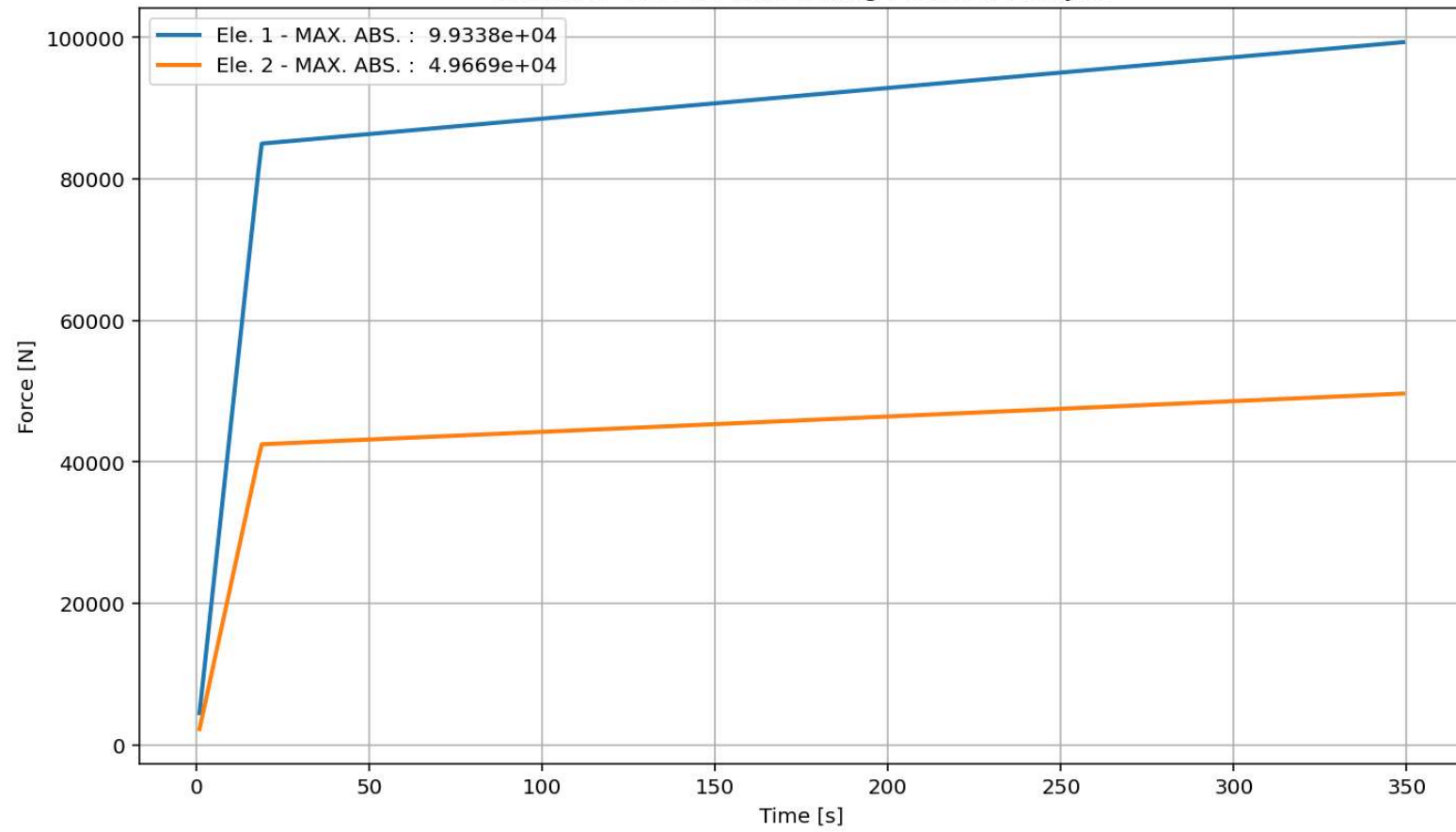




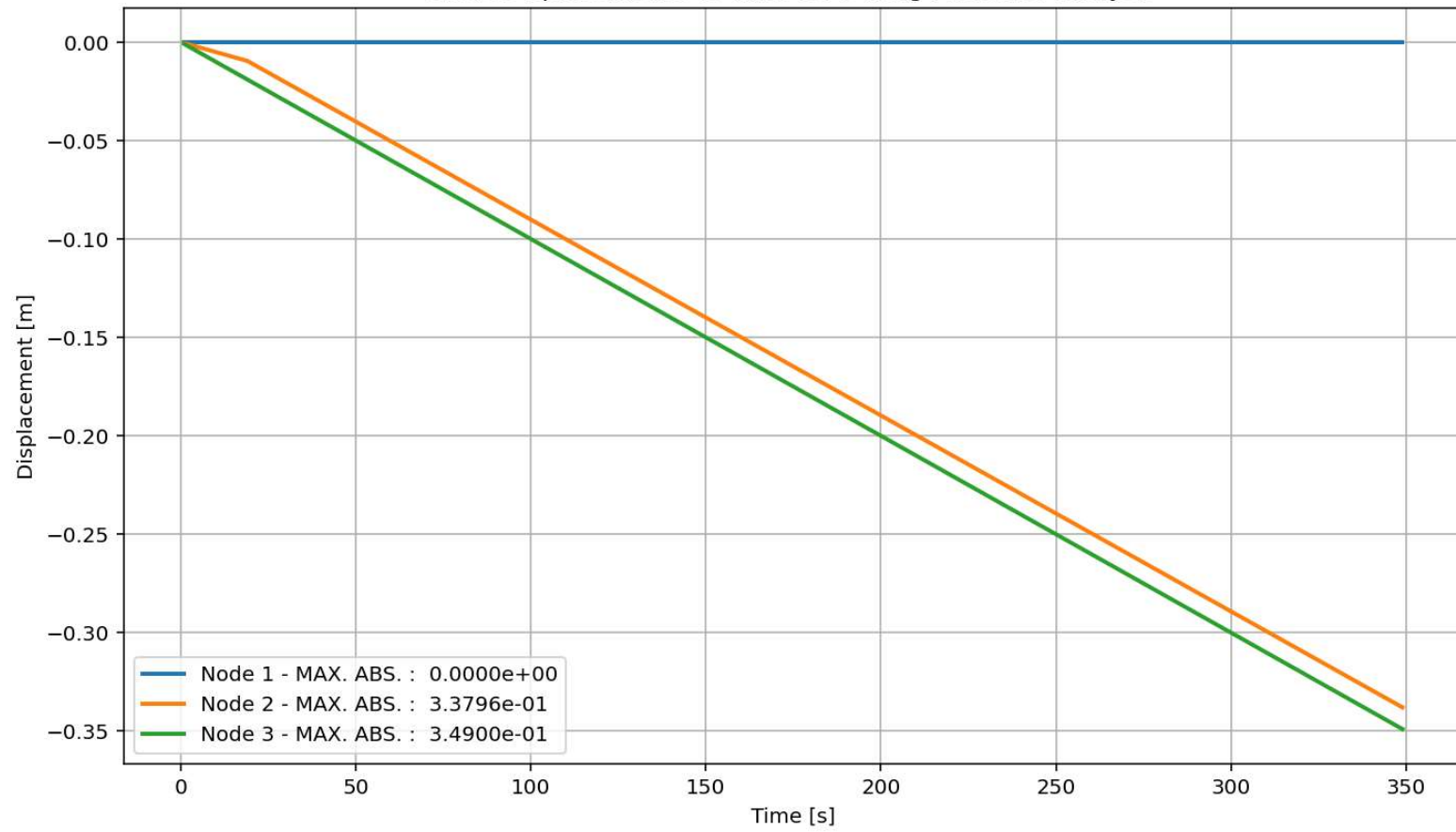
Period of Structure During Pushover Analysis

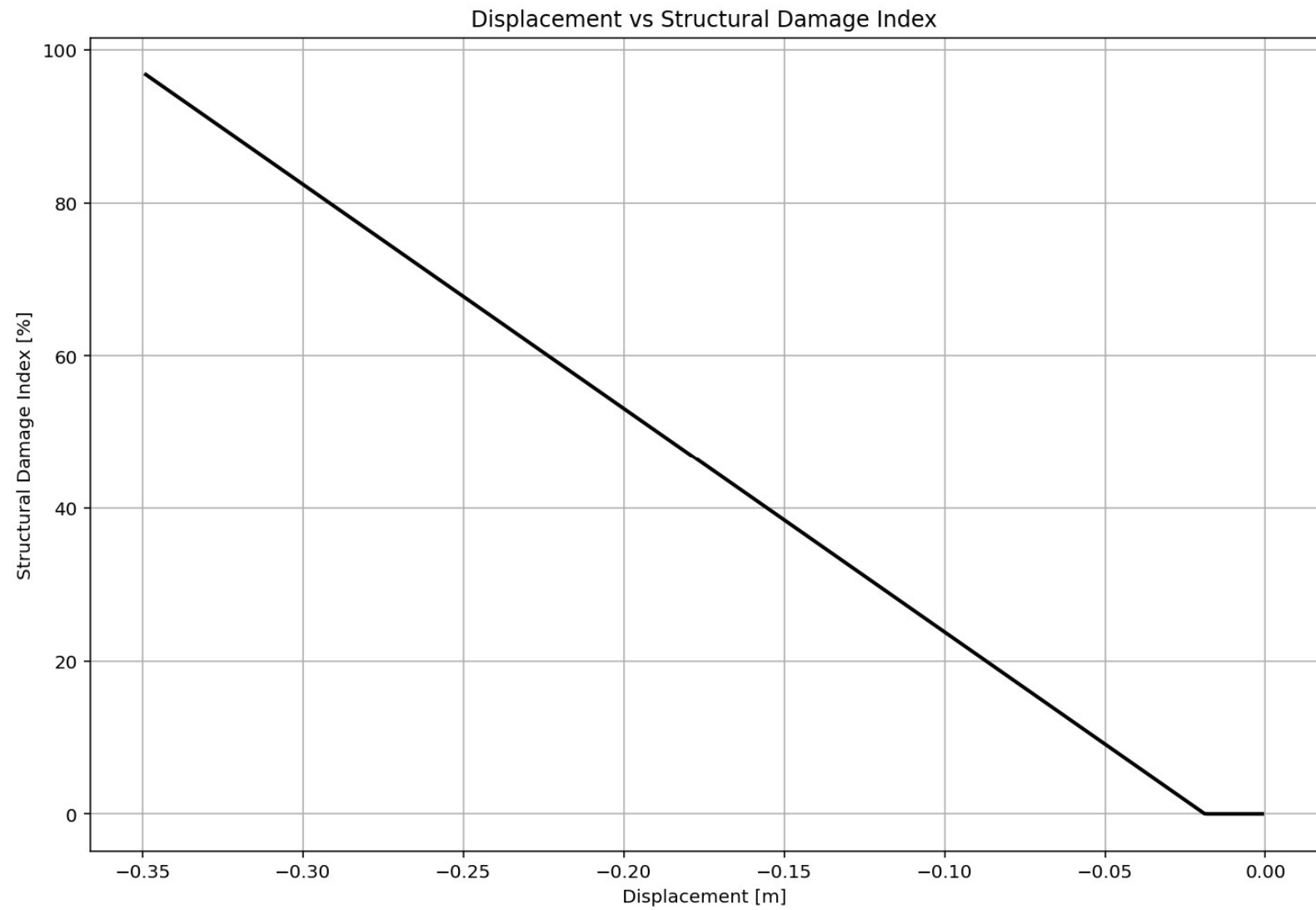


elements force vs Time During Pushover Analysis



Node Displacements vs Time for During Pushover Analysis





Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\Dell\Desktop\OPENSEES_FILES\+EIGHT_ANALY...RE_MDOF_8_ANA\COMPUTE_EFFECTIVE_PROPERTIES_FUN.py

P(t)_SOIL_STRUCTURE_MDOF_8_ANA.py X COMPUTE_EFFECTIVE_PROPERTIES_FUN.py X

```
1  """ EQUIVALENT SDOF SYSTEM DERIVATION VIA DISPLACEMENT-BASED PUSHOVER ANALYSIS
2  # Change MDOF to SDOF System with Displacement Based Design Concept
3  # THIS PROGRAM WRITTEN BY SALAR DELAVAR GHASHGHAEI (QASHQAI)
4  """
5  This script implements a displacement-based pushover transformation,
6  converting a multi-degree-of-freedom (MDOF) system into an equivalent
7  single-degree-of-freedom (SDOF) system for seismic assessment.
8  It calculates effective modal properties-displacement, mass, and
9  stiffness-by weighting element forces and nodal displacements according
10 to a presumed deformed shape.
11 The derived effective period provides a simplified dynamic characteristic
12 for performance-based engineering. The visualizations effectively track the
13 evolution of these equivalent parameters throughout the nonlinear analysis steps.
14 """
15 #-----
16 displacement_X_1 = np.array(list(node_displacements.values())[1]) # DOF 02
17 displacement_X_2 = np.array(list(node_displacements.values())[2]) # DOF 03
18
19
20 ele_force_01 = np.array(list(ele_force.values())[0]) # ELEMENT 01
21 ele_force_02 = np.array(list(ele_force.values())[1]) # ELEMENT 02
22
23 STIFF_X_1 = np.abs(ele_force_01 / displacement_X_1)
24 STIFF_X_2 = np.abs(ele_force_02 / displacement_X_2)
25
26 MX2 = (MASS[0] * np.square(displacement_X_1) +
27        MASS[1] * np.square(displacement_X_2))
28
29 MX = (MASS[0] * np.array(displacement_X_1) +
30       MASS[1] * np.array(displacement_X_2))
31 EFFECTIVE_DISP_X = MX2 / np.abs(MX)
32
33 EFFECTIVE_MASS_X = np.abs(MX) / EFFECTIVE_DISP_X
34
35
```

Effective Displacement - Median: 0.16588

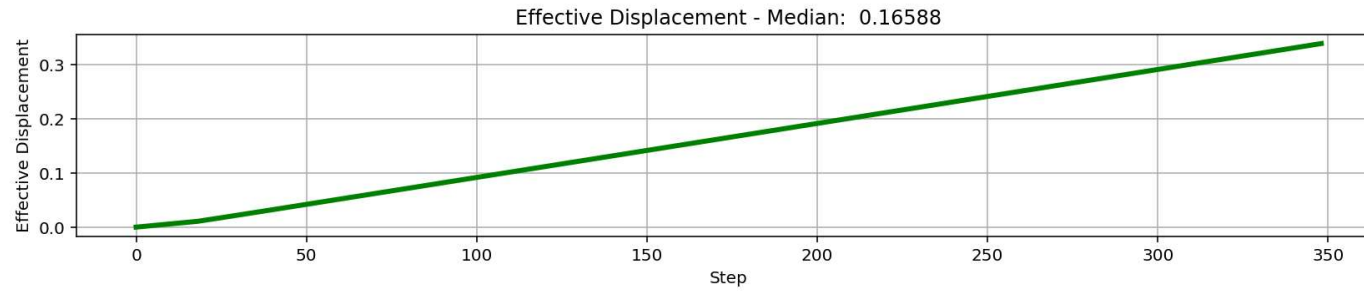
Effective Mass - Median: 4998.29864

Effective Stiffness - Median: 829950.70654

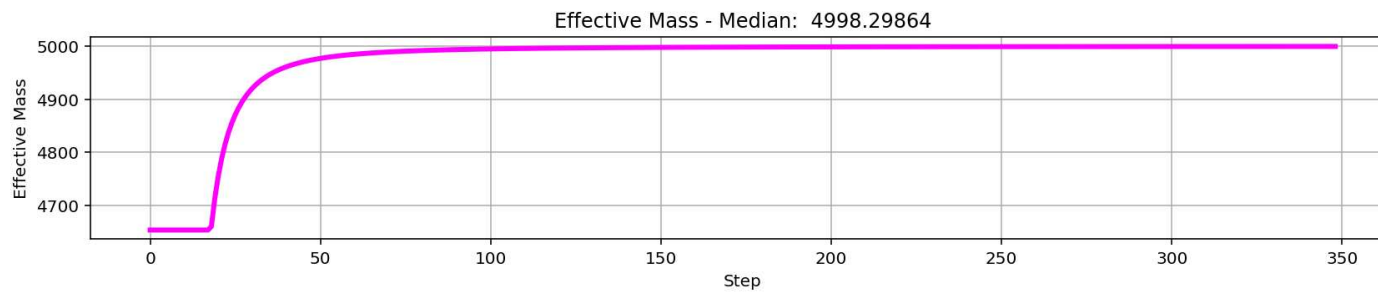
Effective Period - Median: 0.48768

IPython Console Files Help Variable Explorer Debugger Plots History

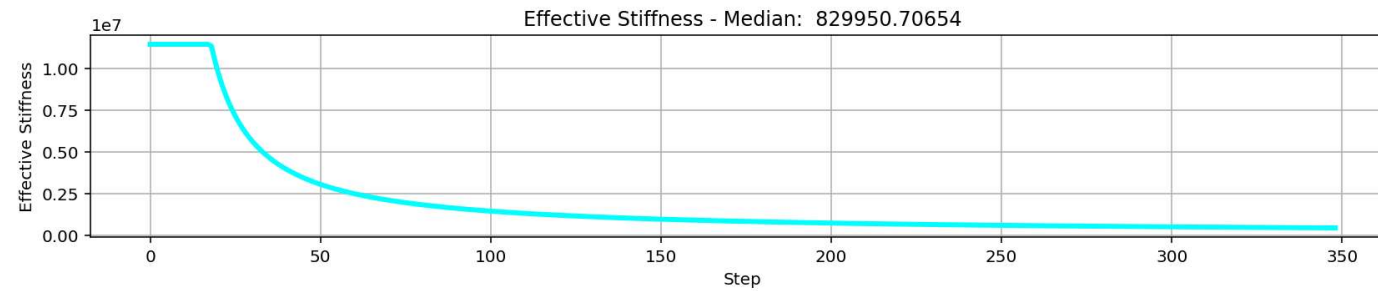
Inline Conda: anaconda3 (Python 3.12.7) ✓ LSP: Python Line 1, Col 1 UTF-8 CRLF RW Mem 56%



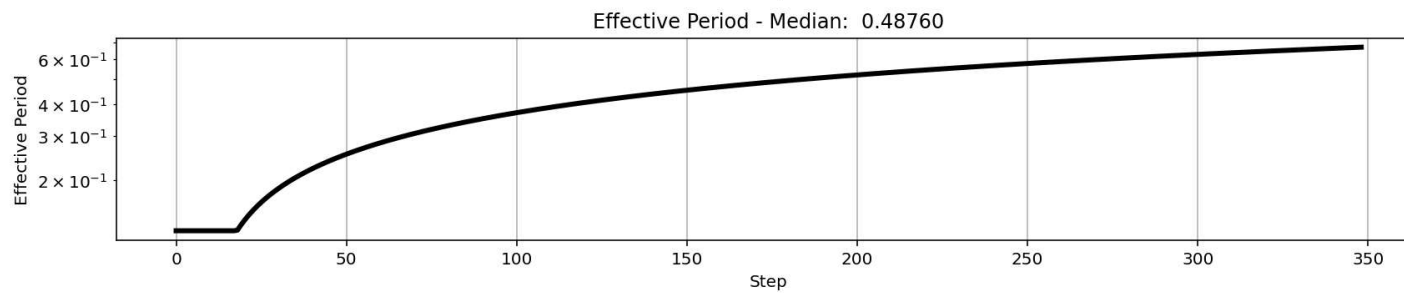
$$\Delta_d = \frac{\sum_{i=1}^n m_i \Delta_i^2}{\sum_{i=1}^n m_i \Delta_i}$$



$$m_{eff} = \frac{\sum_{i=1}^n m_i \Delta_i}{\Delta_d}$$



$$k_{eff} = \frac{\sum_{i=1}^n k_i \Delta_i}{\Delta_d}$$



$$T_{eff} = \frac{2\pi}{\sqrt{\frac{k_{eff}}{m_{eff}}}}$$

CYCLIC DISPLACEMENT ANALYSIS

Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\ DELL\Desktop\OPENSEES_FILES\+EIGHT_ANALY...N_CAPACITY_INDEX\22_MDOF_8_ANA\P(t)_MDOF_8_ANA.py

P(t)_MDOF_8_ANA.py X

```
284 if ANAL_TYPE == 'CYCLIC_DISPLACEMENT': # STATIC TIME-HISTORY ANALYSIS
285     IDctrlDOF = 1 # 1: Horizontal Displacement - 2: Vertical Displacement
286     DMAX = -0.35 # [m] Max. Displacement
287     n_points = 10000
288     # 4. CYCLIC DISPLACEMENT PROTOCOL
289     # Key strain points (same logic as your protocol)
290     key_disp = np.array([
291         0.0,
292         0.1*DMAX, -0.1*DMAX,
293         0.5*DMAX, -0.5*DMAX,
294         0.8*DMAX, -0.8*DMAX,
295         DMAX, -DMAX,
296         0.1*DMAX, -0.1*DMAX,
297         0.5*DMAX, -0.5*DMAX,
298         0.8*DMAX, -0.8*DMAX,
299         DMAX, -DMAX,
300         0.0
301     ])
302
303     # Generate 1000-point displacement protocol
304     disp_protocol = np.interp(
305         np.linspace(0, len(key_disp) - 1, n_points),
306         np.arange(len(key_disp)),
307         key_disp
308     )
309
310     ops.analysis('Static')
311     STEP = 0.0
312     for target_disp in disp_protocol:
313         current_disp = ops.nodeDisp(center_node, 1) # DISPLACEMENT APPLIED IN MIDDLE N...
314         dU = target_disp - current_disp
315         ops.integrator('DisplacementControl', center_node, IDctrlDOF, dU)
316         OK = ops.analyze(1)
317         S02.ANALYSIS(OK, 1, MAX_TOLERANCE, MAX_ITERATIONS)
```

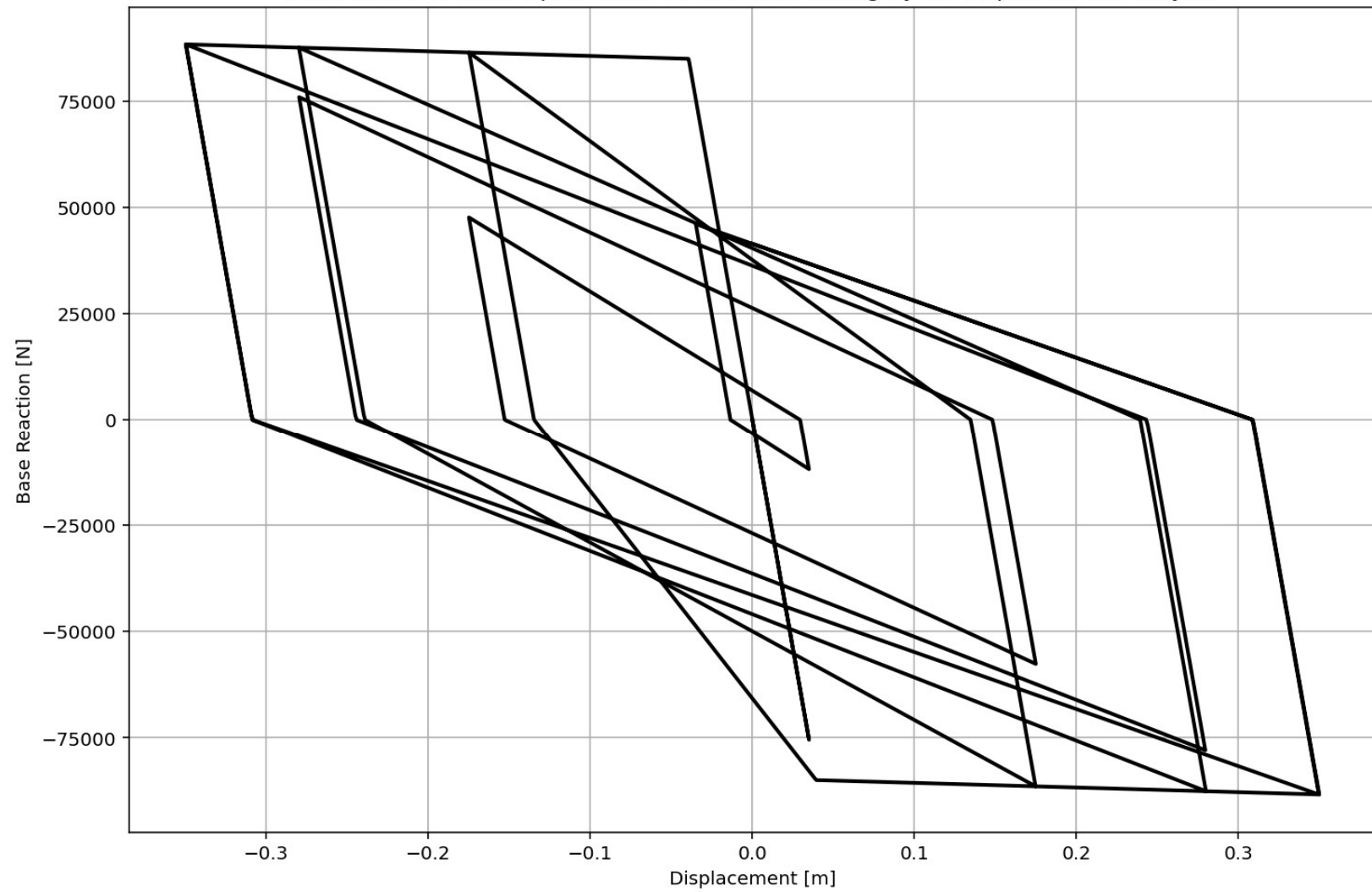
Base Reaction and Displacement of Structure During Cyclic-Displacement Analysis



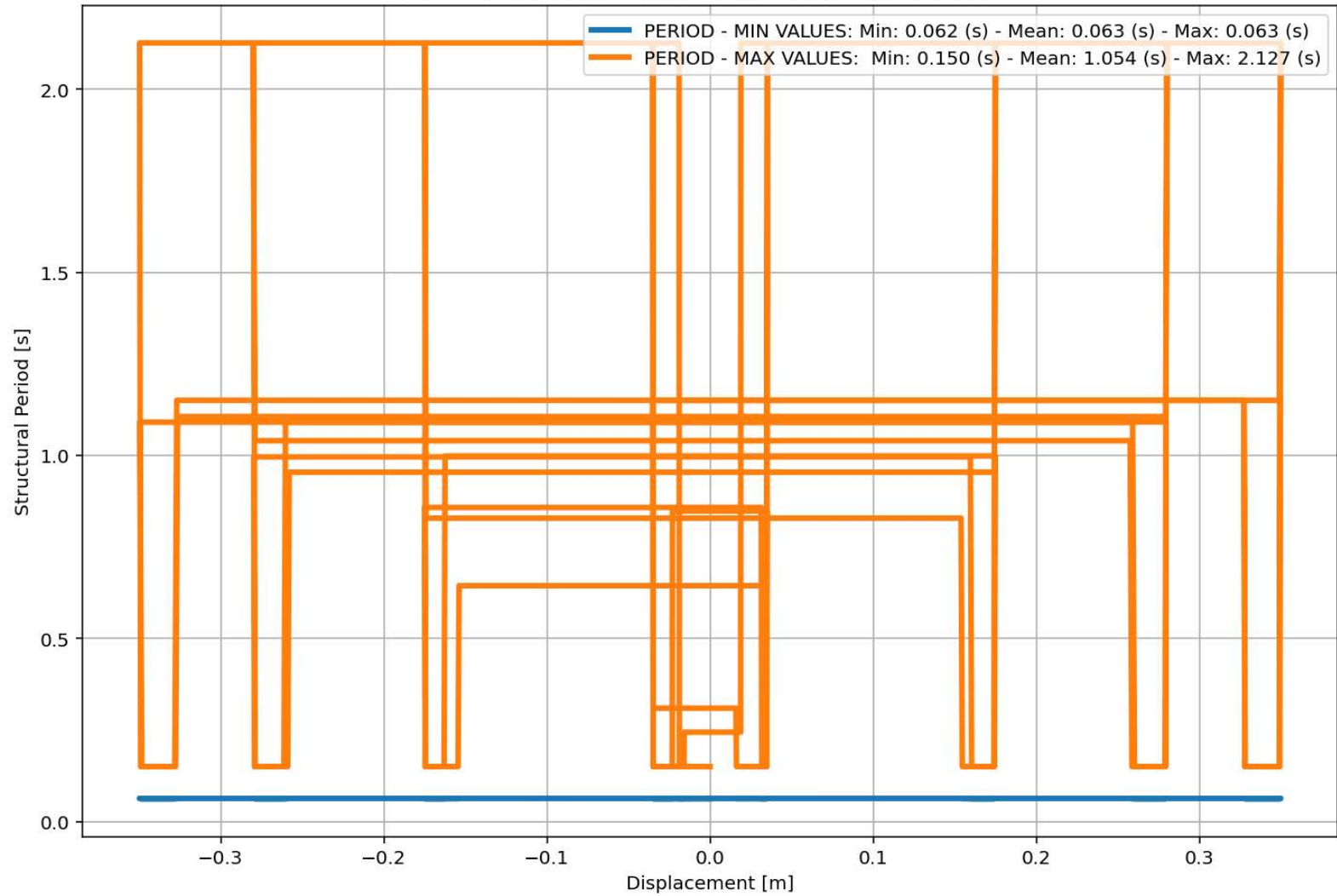
IPython Console Files Help Variable Explorer Debugger Plots History

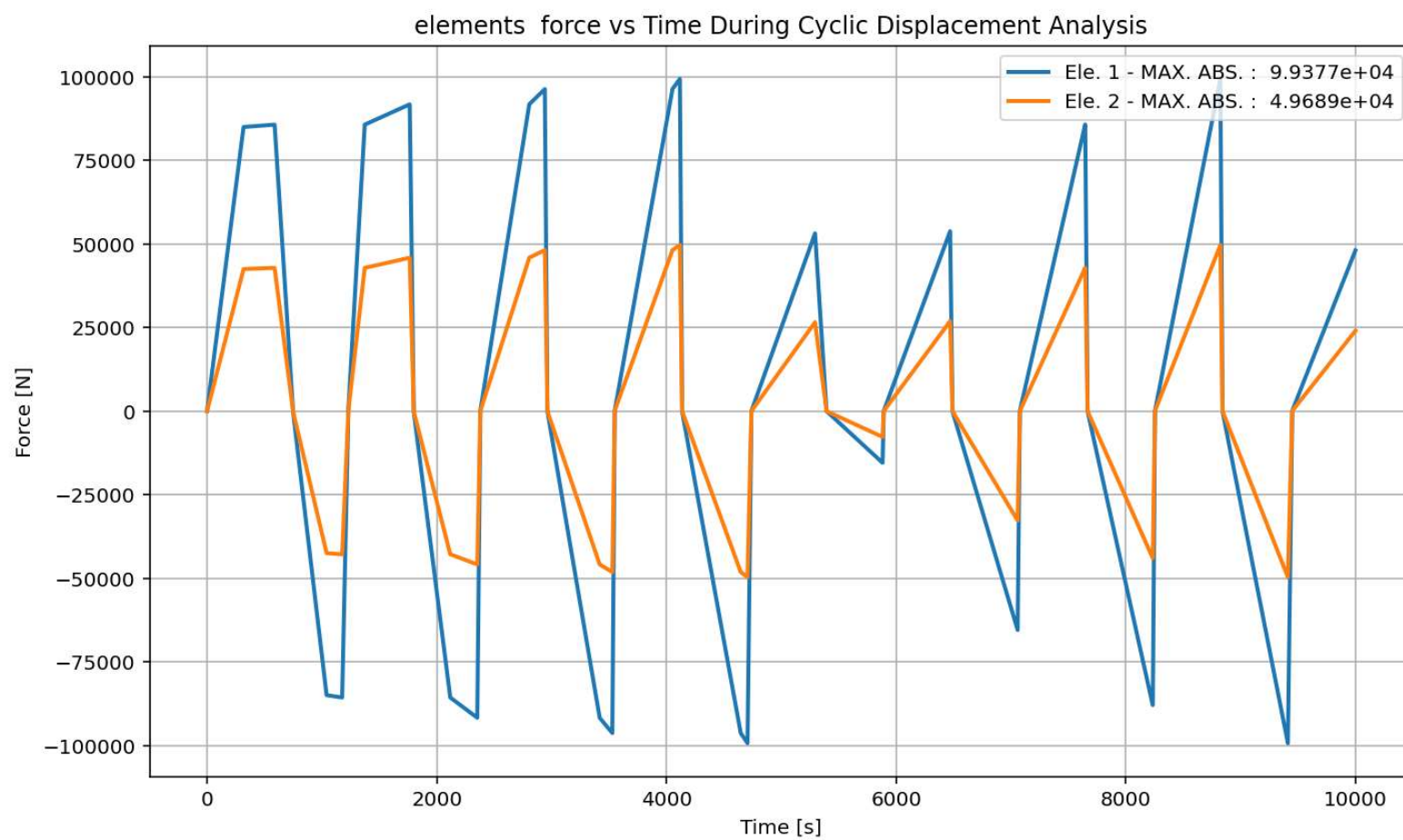
Inline Conda: anaconda3 (Python 3.12.7) ✓ LSP: Python Line 1307, Col 23 UTF-8 CRLF RW Mem 45%

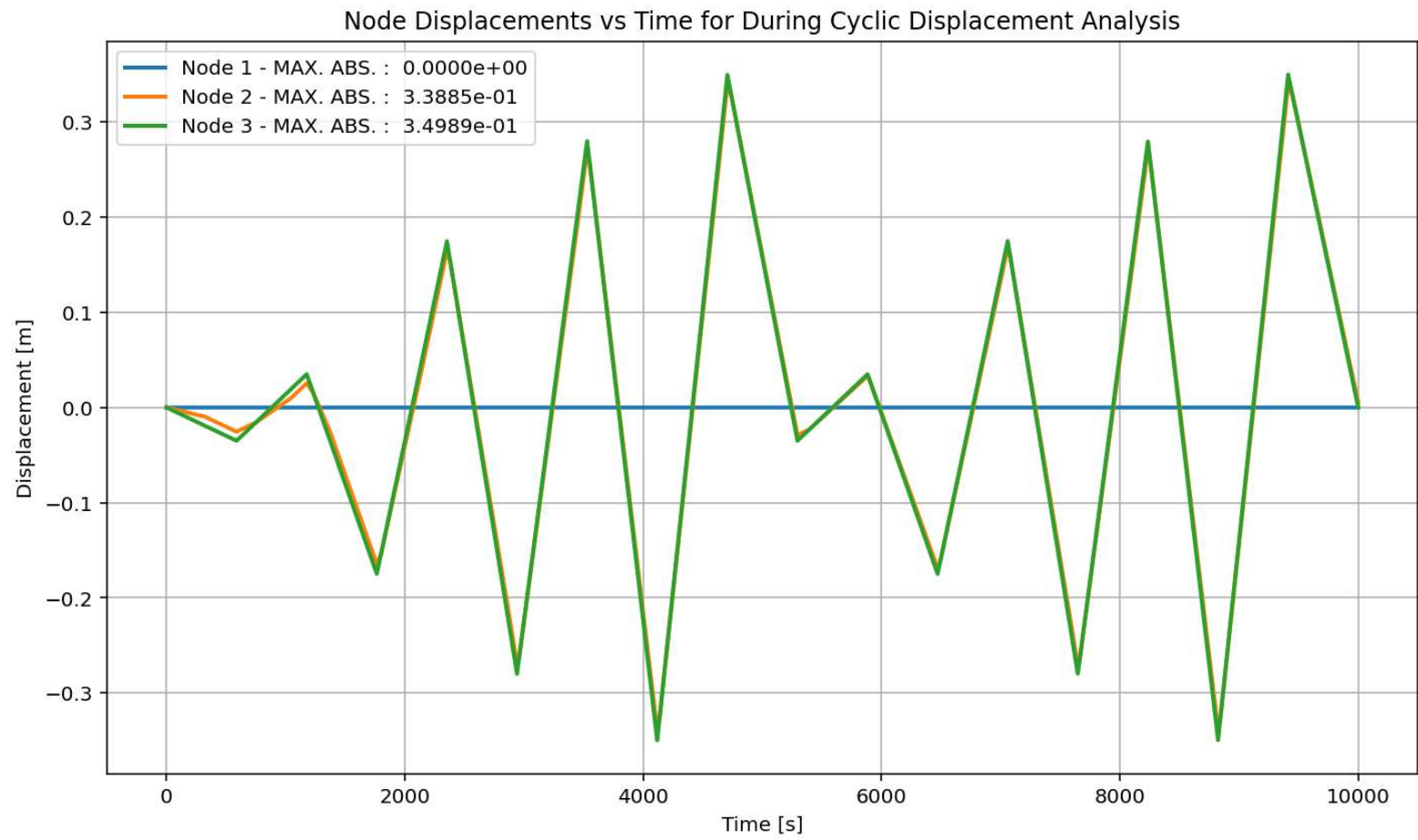
Base Reaction and Displacement of Structure During Cyclic-Displacement Analysis



Period of Structure During Cyclic Displacement Analysis

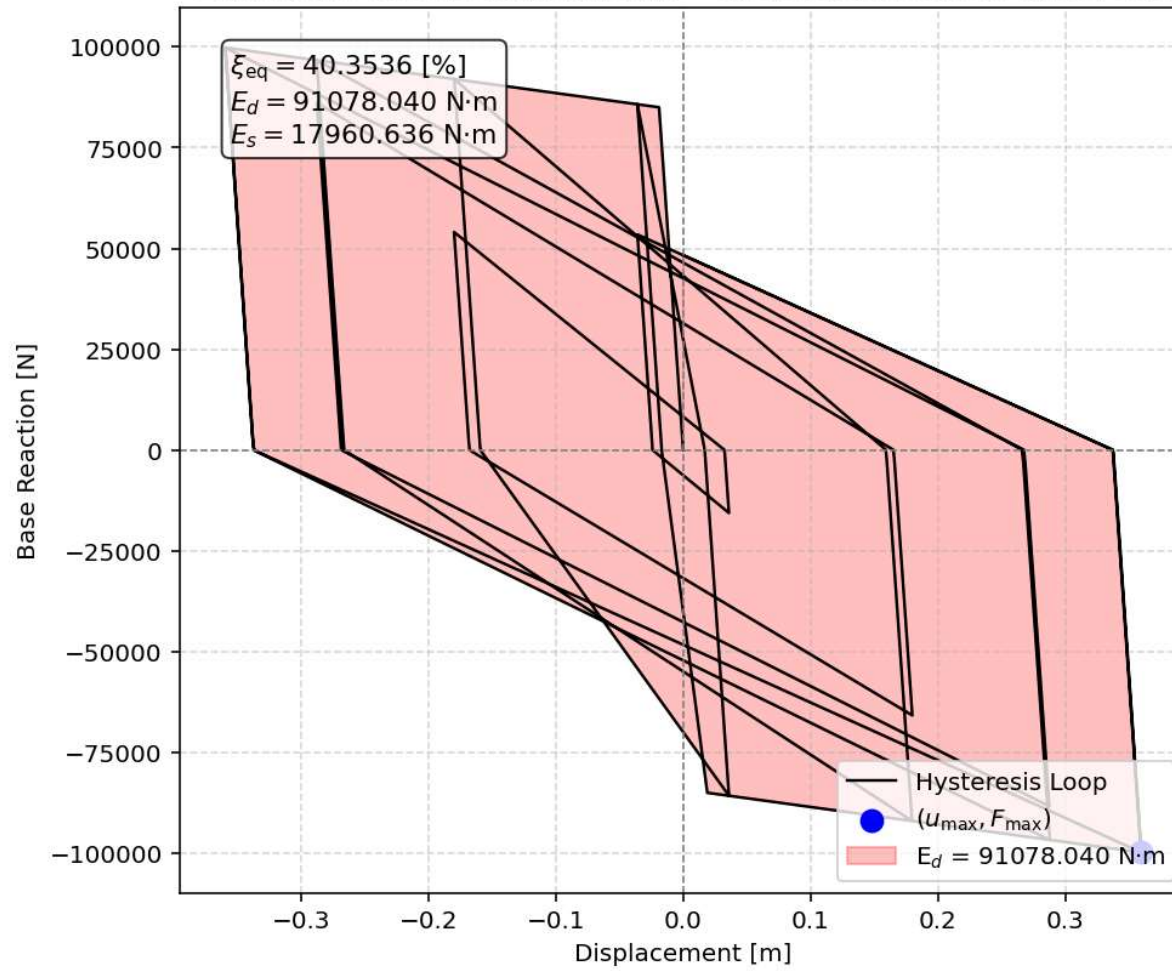








Equivalent viscous damping ratio - Cyclic Displacement Hysteresis



STATIC TIME-HISTORY WITH EXTERNAL TIME-DEPENDENT LOADING ANALYSIS

Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\Dell\Desktop\OPENSEES_FILES\HEIGHT_ANALY...N_CAPACITY_INDEX\22_MDOF_8_ANA\P(t)_MDOF_8_ANA.py

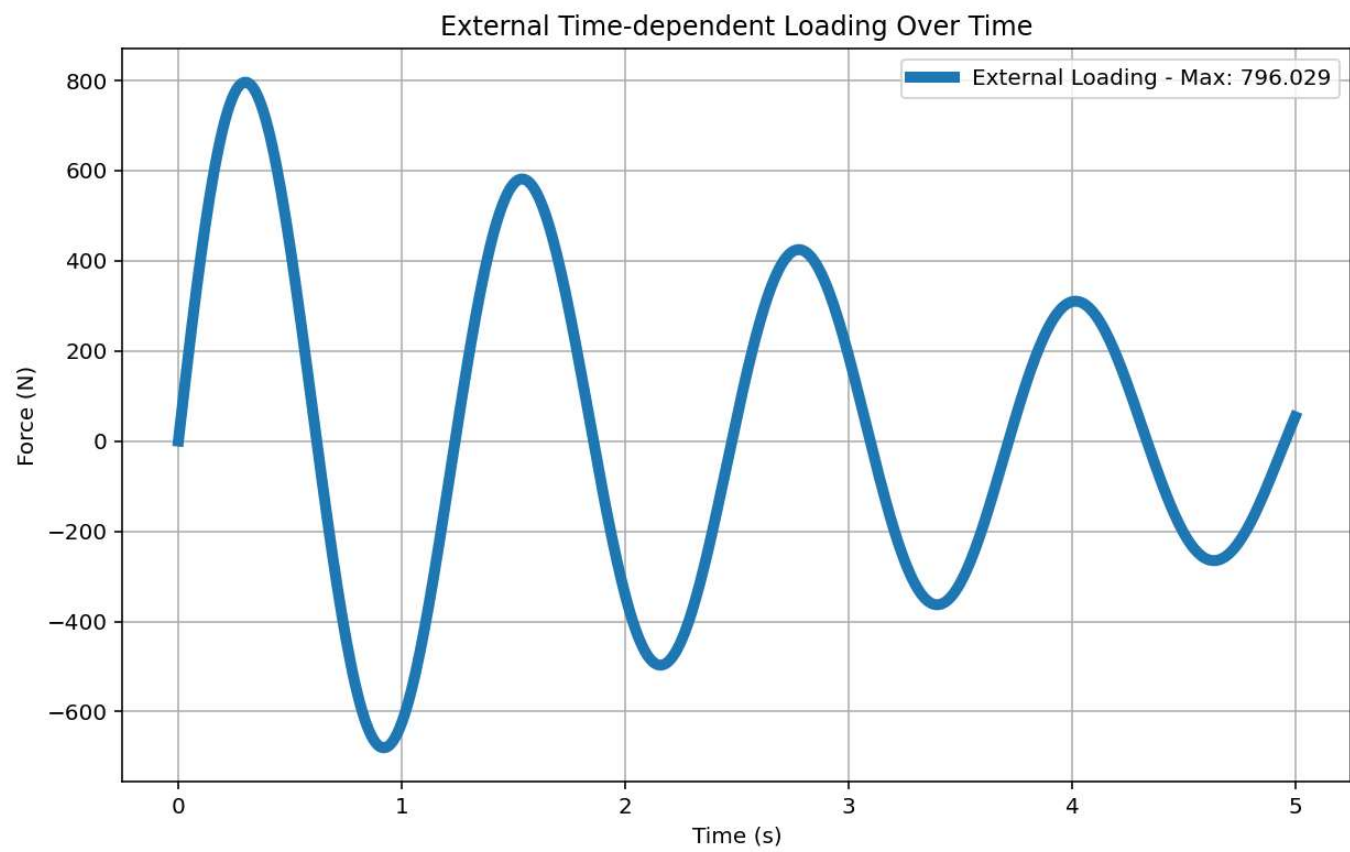
P(t)_MDOF_8_ANA.py

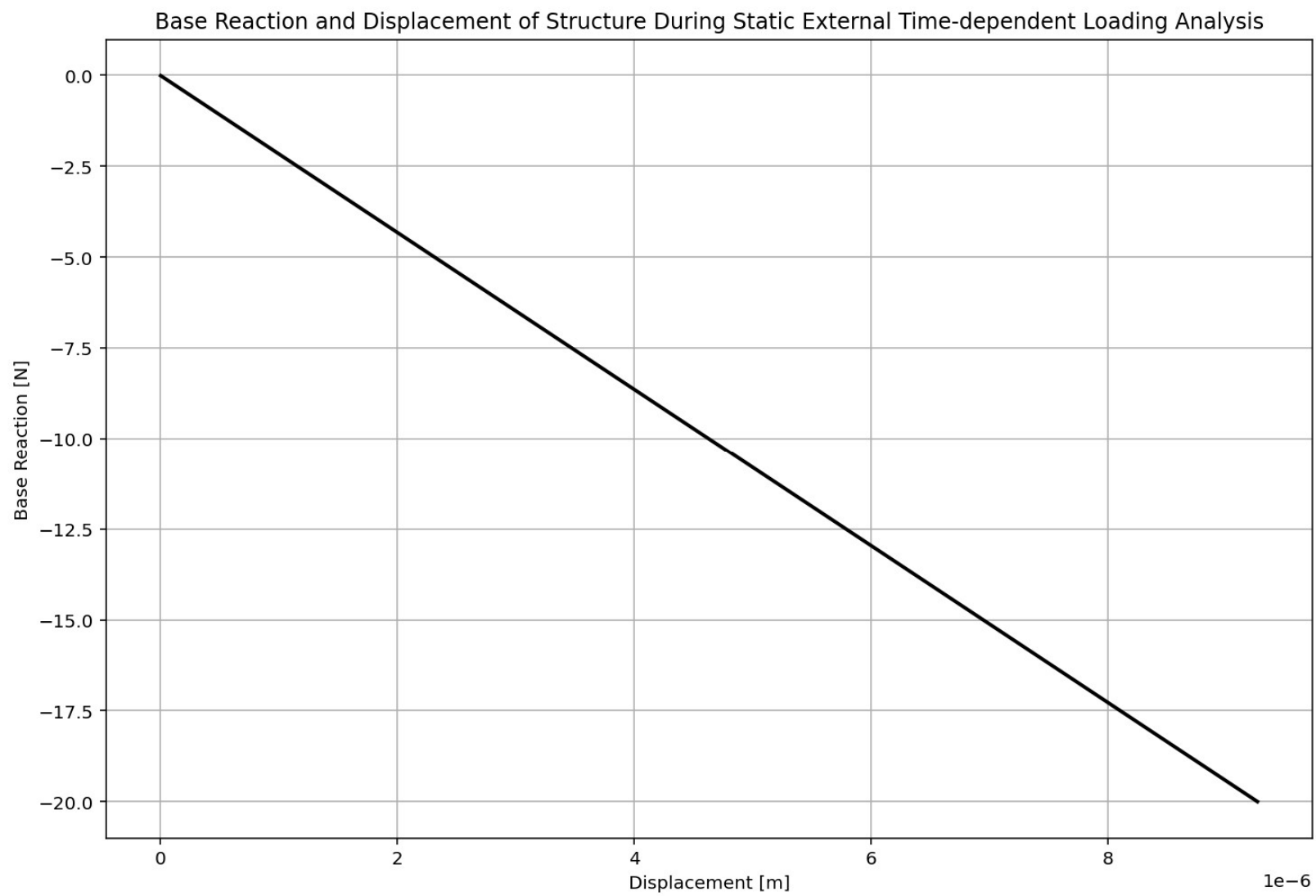
```
349 if ANAL_TYPE == 'STATIC EXTERNAL TIME-DEPENDENT LOADING': # STATIC TIME-HISTORY ANALYSIS
350     %% DEFINE EXTERNAL TIME-DEPENDENT LOADING PROPERTIES
351     # IN HERE ANALYSIS TIME AND DURATION ARE LOAD STEPS
352     duration = 20.0 # [s] Analysis duration
353     dt = 0.01 # [s] Time step
354     DT = dt # [s] Time step
355     DT_time = 5.0 # [s] Total external Load Analysis Durations [*****]
356     force_amplitude = 5960.0 # [N] Amplitude Force
357     omega_DT = 5.0715 # [rad/s] Natural angular frequency
358     time_steps = int(duration/dt)
359
360     # Check function
361     def CHECK_FUN(DT_time ,duration):
362         if DT_time > duration:
363             print('\n\nAnalysis Duration Must be greater than External Load Duration\n\n')
364             exit()
365         return -1
366
367     CHECK_FUN(DT_time ,duration)
368     def EXTERNAL_TIME_DEPENDENT(force_amplitude, omega_DT, DT, DT_time): # P(t) = P0 sin(omega*t)
369         import numpy as np
370         import matplotlib.pyplot as plt
371         # External Load Durations
372         num_steps = int(DT_time / DT)
373         load_time = np.linspace(0, DT_time, num_steps)
374         target_frequency = 1.0 * omega_DT # Target excitation frequency
375         DT_load = force_amplitude * np.sin(target_frequency * load_time)
376         # Plot External Time-dependent Loading
377         plt.figure(figsize=(10, 6))
378         plt.plot(load_time, DT_load, label=f'External Loading - Max: {np.max(DT_load)}')
379         plt.title('External Time-dependent Loading Over Time')
380         plt.xlabel('Time (s)')
381         plt.ylabel('Force (N)')
382         plt.grid(True)
```

External Time-dependent Loading Over Time

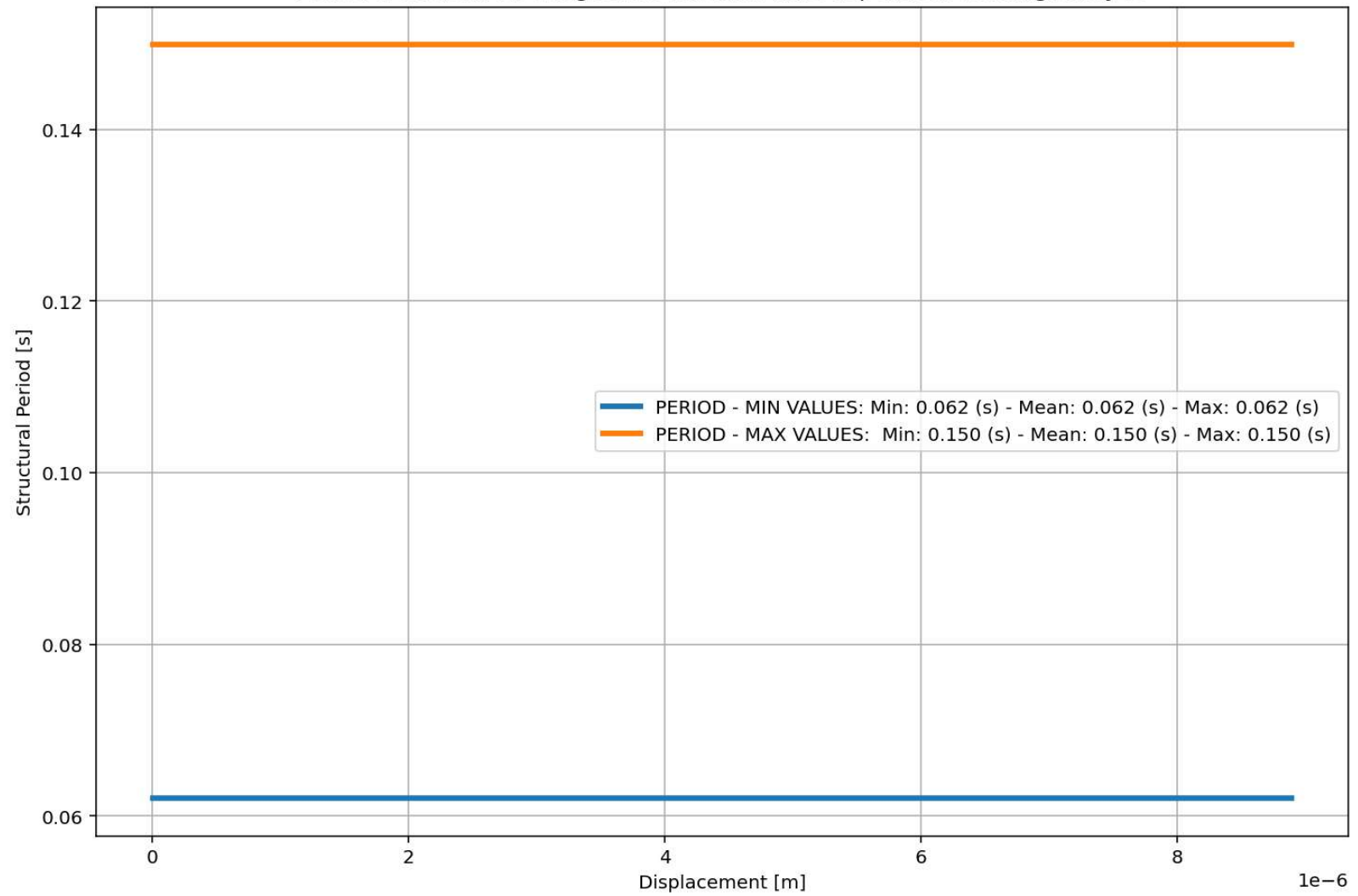
IPython Console Files Help Variable Explorer Debugger Plots History

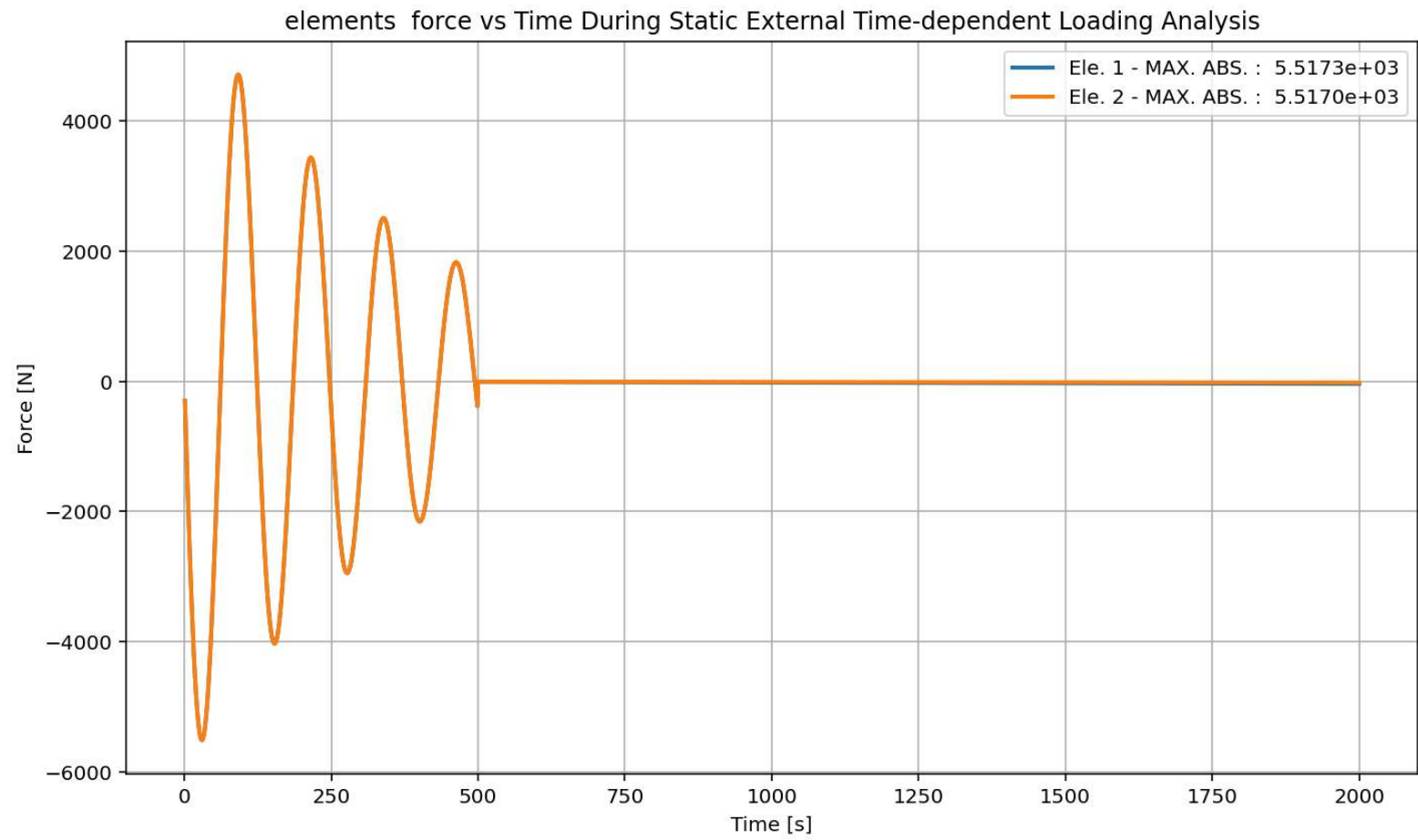
Inline Conda: anaconda3 (Python 3.12.7) LSP: Python Line 1307, Col 23 UTF-8 CRLF RW Mem 45%

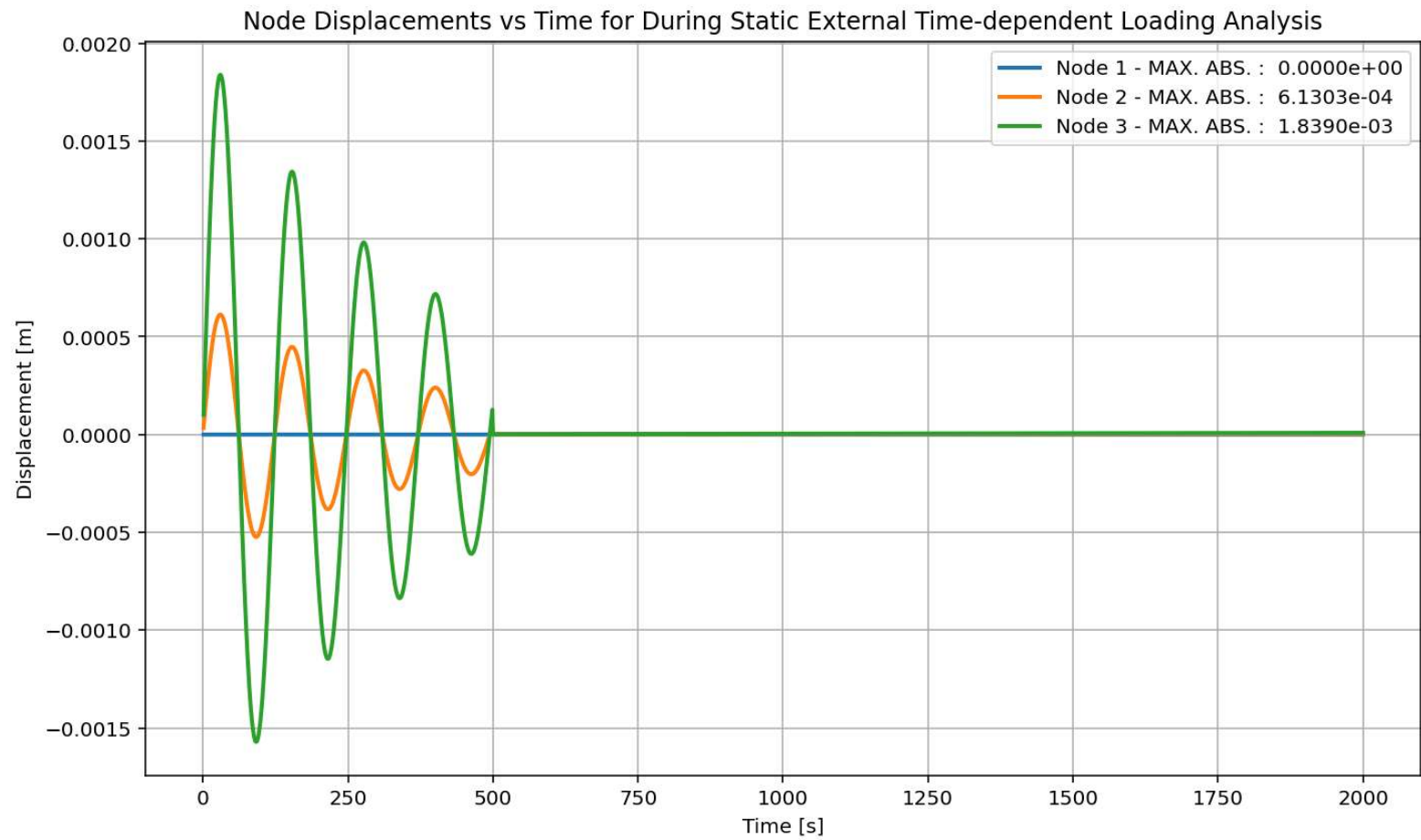


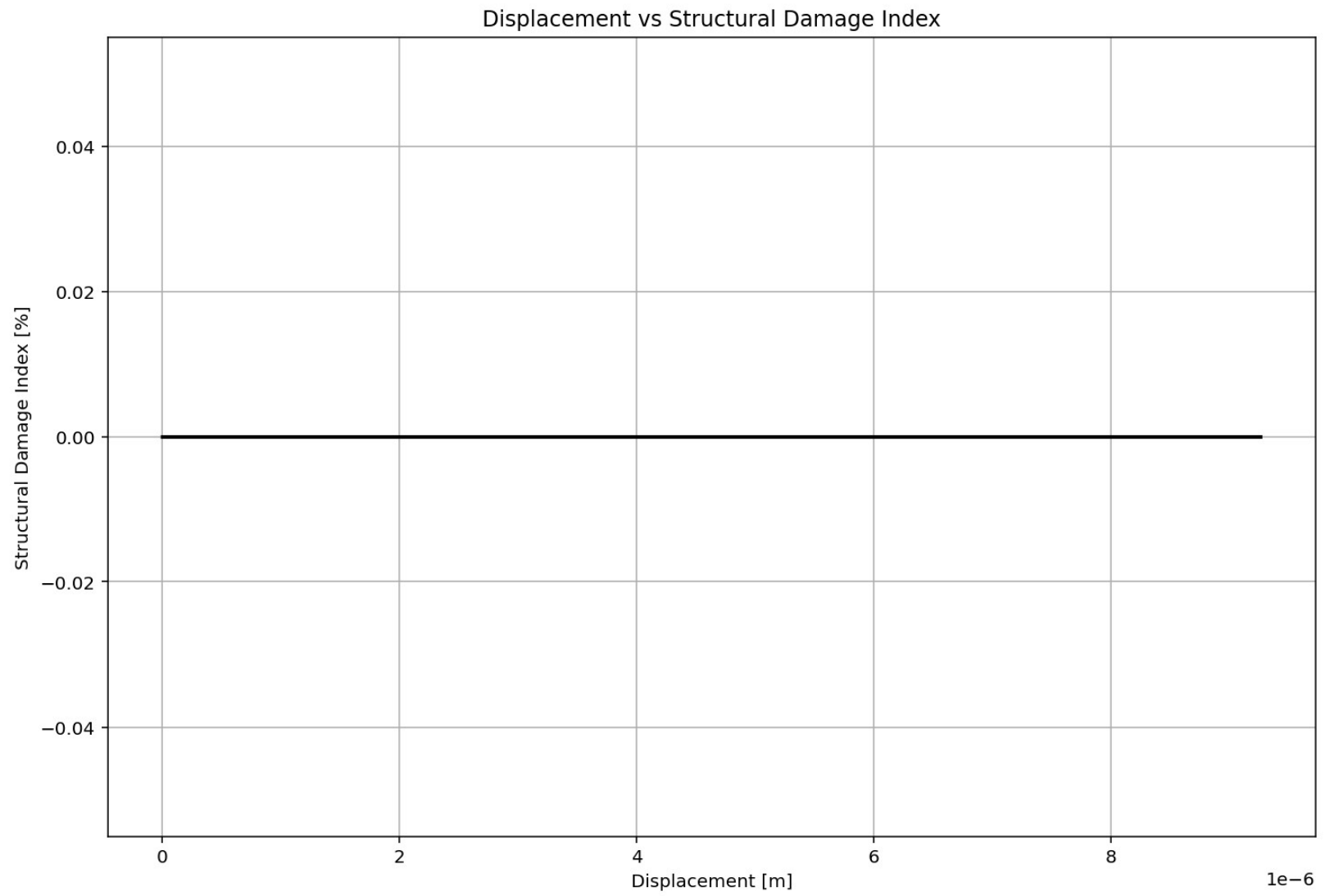


Period of Structure During Static External Time-dependent Loading Analysis









DYNAMIC TIME-HISTORY WITH EXTERNAL TIME-DEPENDENT LOADING ANALYSIS

Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\De\l\Desktop\OPENSEES_FILES\+EIGHT_ANALY...N_CAPACITY_INDEX\22_MDOF_8_ANA\P(t)_MDOF_8_ANA.py

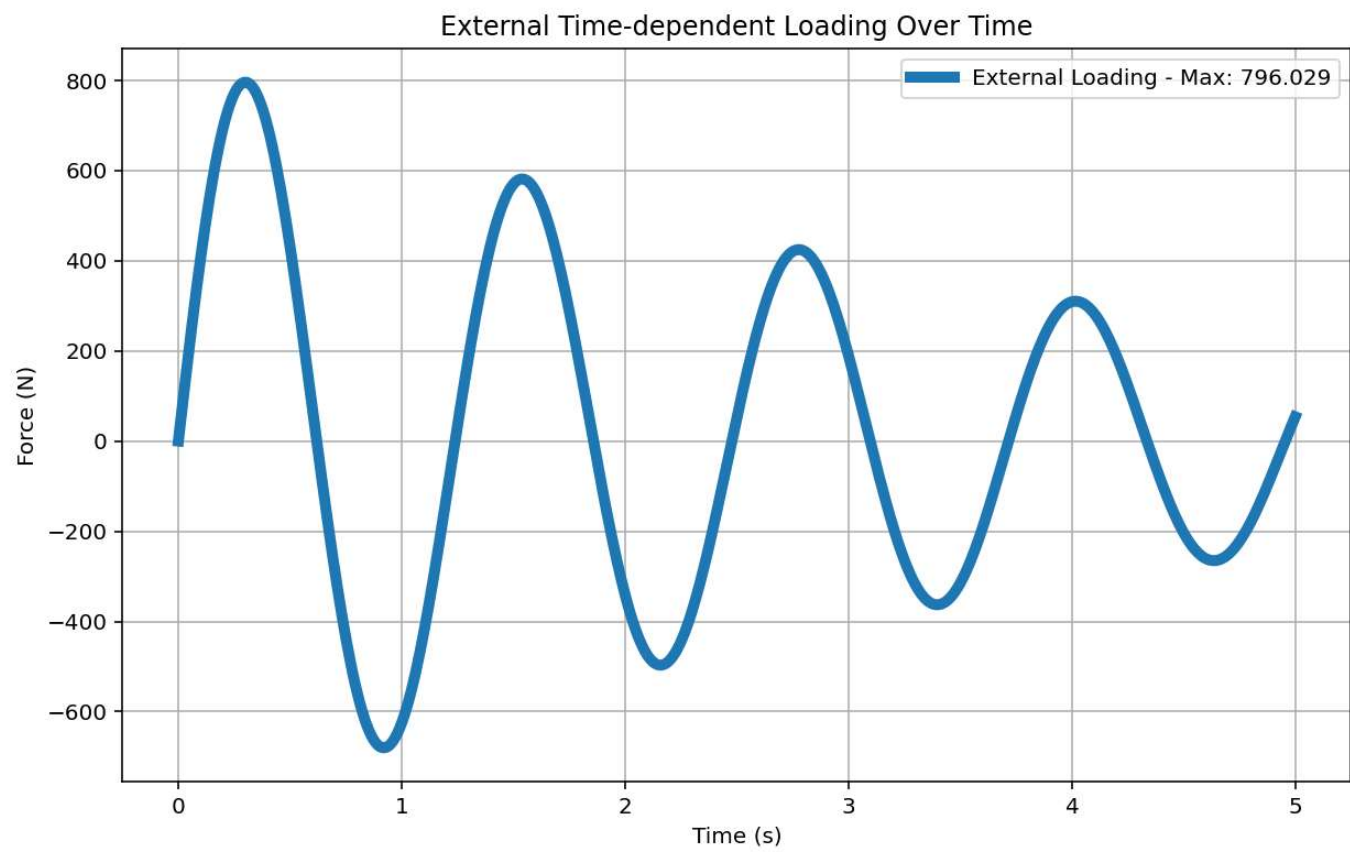
P(t)_MDOF_8_ANA.py X

```
462 if ANAL_TYPE == 'DYNAMIC EXTERNAL TIME-DEPENDENT LOADING': # DYNAMIC TIME-HISTORY ANAL
463     %% DEFINE EXTERNAL TIME-DEPENDENT LOADING PROPERTIES
464     duration = 20.0          # [s] Analysis duration
465     dt = 0.01               # [s] Time step
466     DT = dt                 # [s] Time step
467     DT_time = 5.0           # [s] Total external Load Analysis Durations [*****]
468     force_amplitude = 5960.0 # [N] Amplitude Force
469     omega_DT = 5.0715       # [rad/s] Natural angular frequency
470     DR = 0.03               # Damping Ratio
471
472     # Check function
473     def CHECK_FUN(DT_time ,duration):
474         if DT_time > duration:
475             print('\n\nAnalysis Duration Must be greater than External Load Duration\n')
476             exit()
477         return -1
478
479     CHECK_FUN(DT_time ,duration)
480     def EXTERNAL_TIME_DEPENDENT(force_amplitude, omega_DT, DT, DT_time): # P(t) = P0 sin(omega_DT * t)
481         import numpy as np
482         import matplotlib.pyplot as plt
483         # External Load Durations
484         num_steps = int(DT_time / DT)
485         load_time = np.linspace(0, DT_time, num_steps)
486         target_frequency = 1.0 * omega_DT # Target excitation frequency
487         DT_load = force_amplitude * np.sin(target_frequency * load_time)
488         # Plot External Time-dependent Loading
489         plt.figure(figsize=(10, 6))
490         plt.plot(load_time, DT_load, label=f'External Loading - Max: {np.max(DT_Load):.1f}')
491         plt.title('External Time-dependent Loading Over Time')
492         plt.xlabel('Time (s)')
493         plt.ylabel('Force (N)')
494         plt.grid(True)
495         plt.legend()
```

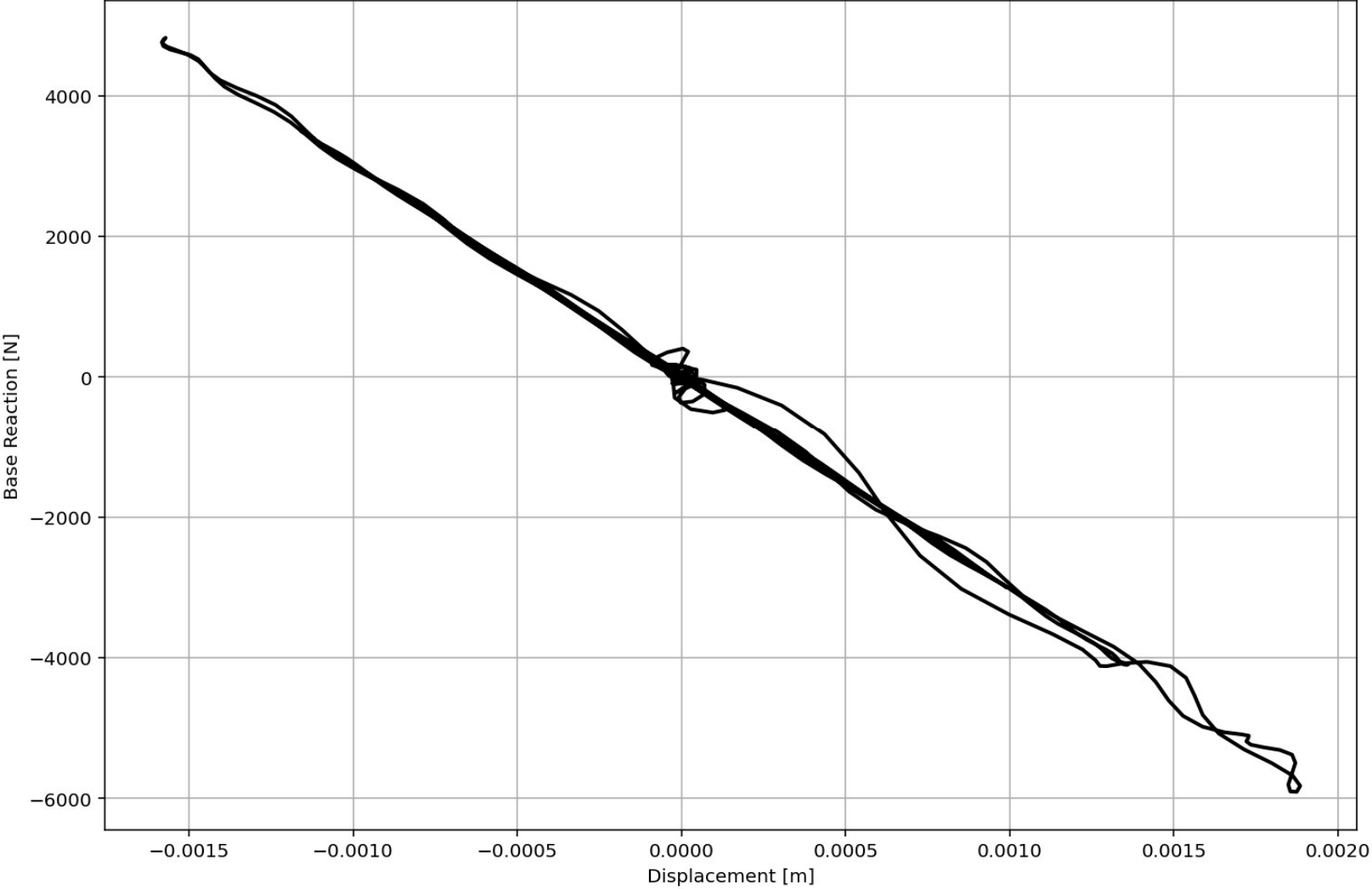
External Time-dependent Loading Over Time

IPython Console Files Help Variable Explorer Debugger Plots History

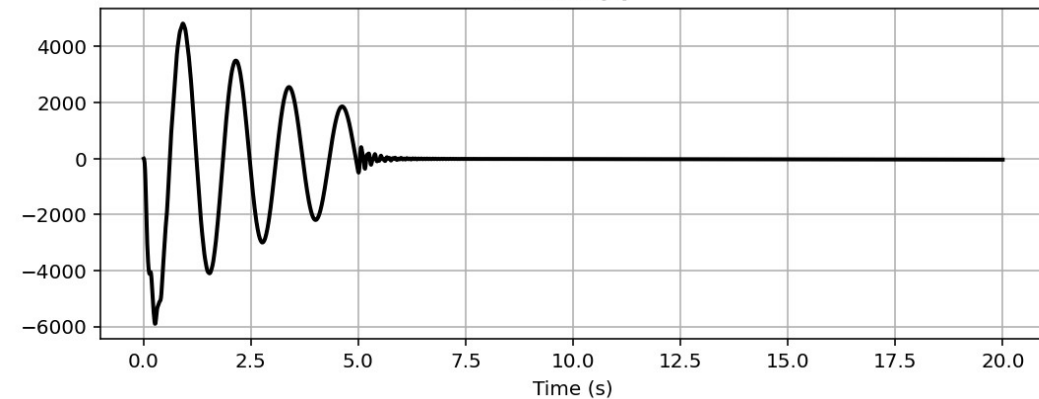
Inline Conda: anaconda3 (Python 3.12.7) ✓ LSP: Python Line 1307, Col 23 UTF-8 CRLF RW Mem 46%



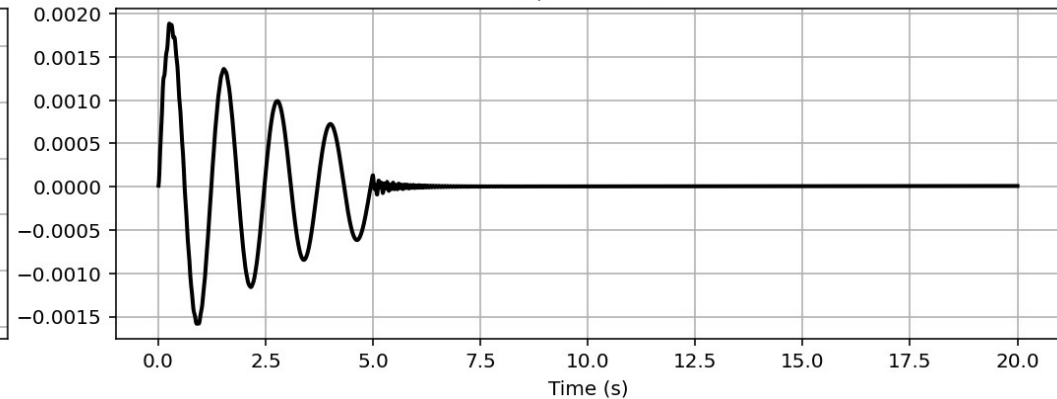
Base Reaction and Displacement of Structure During Dynamic External Time-dependent Loading Analysis



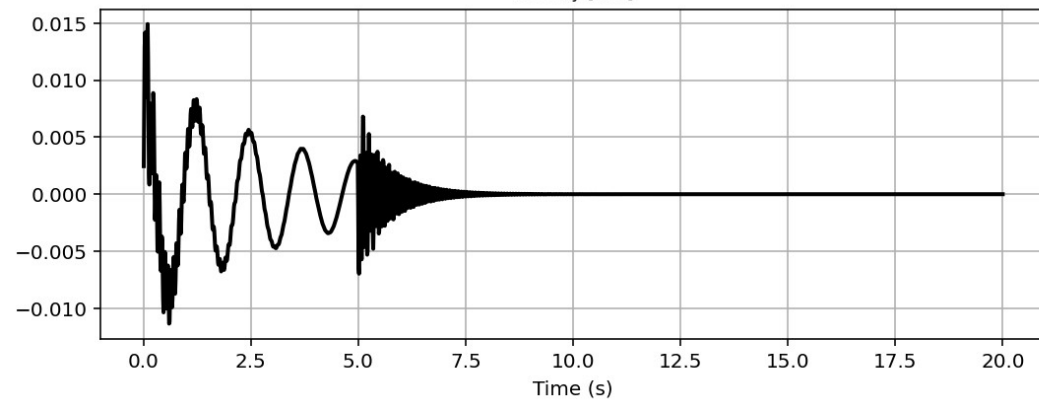
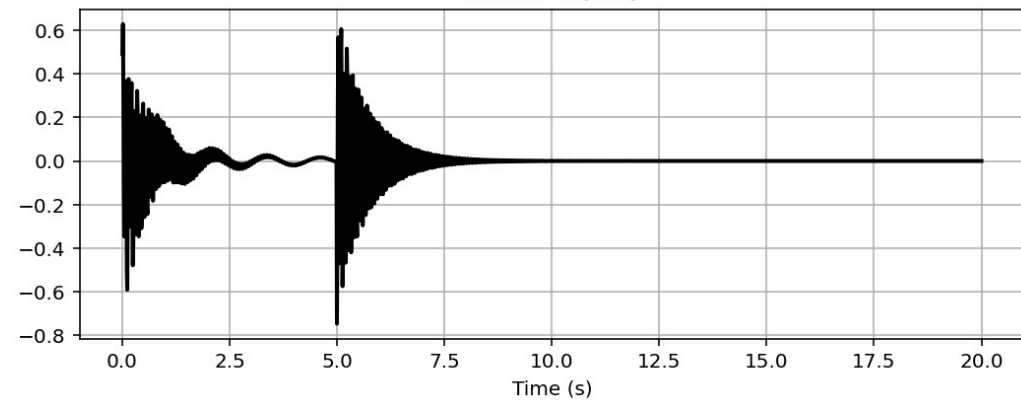
Reaction [N]



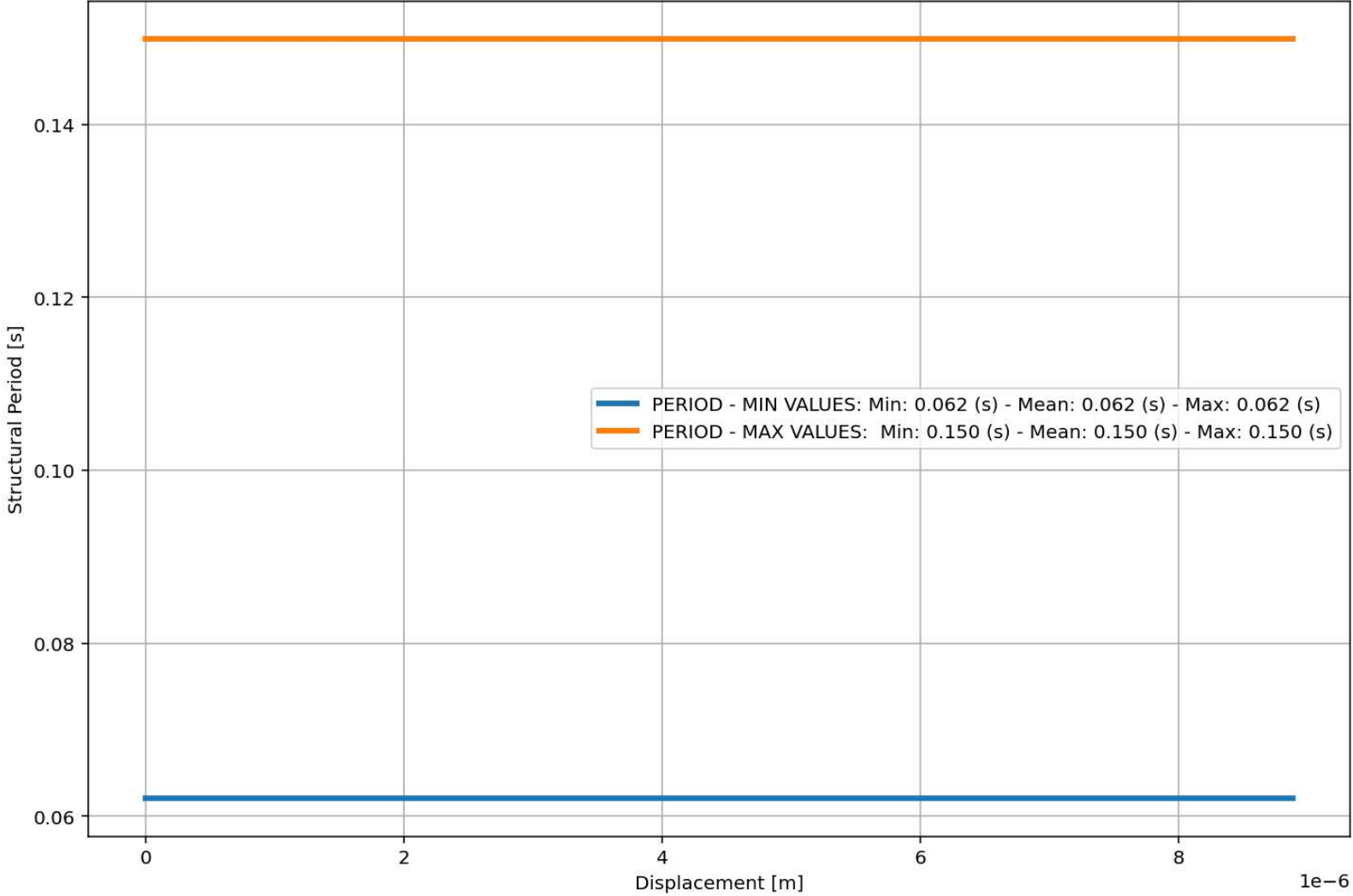
Displacement [m]

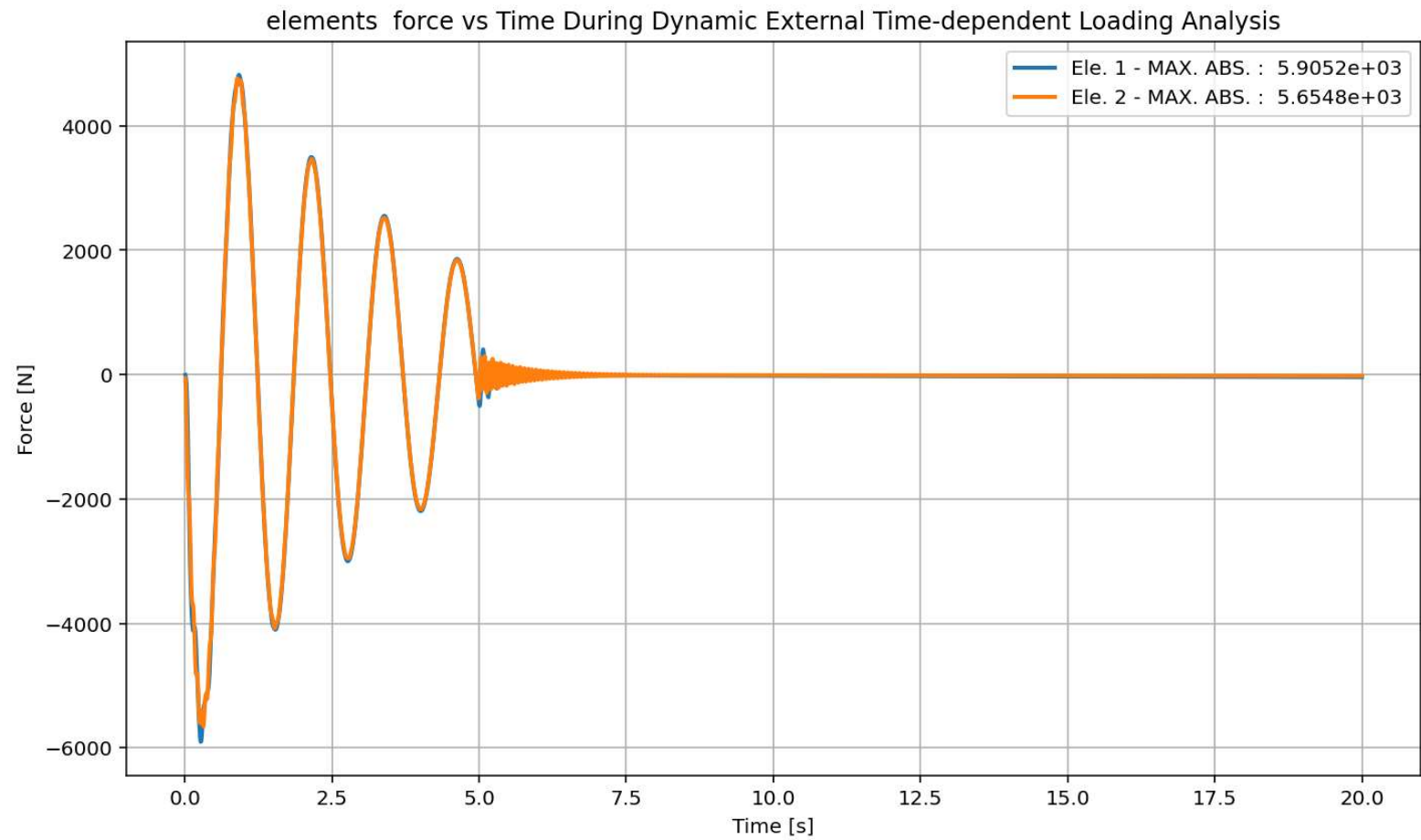


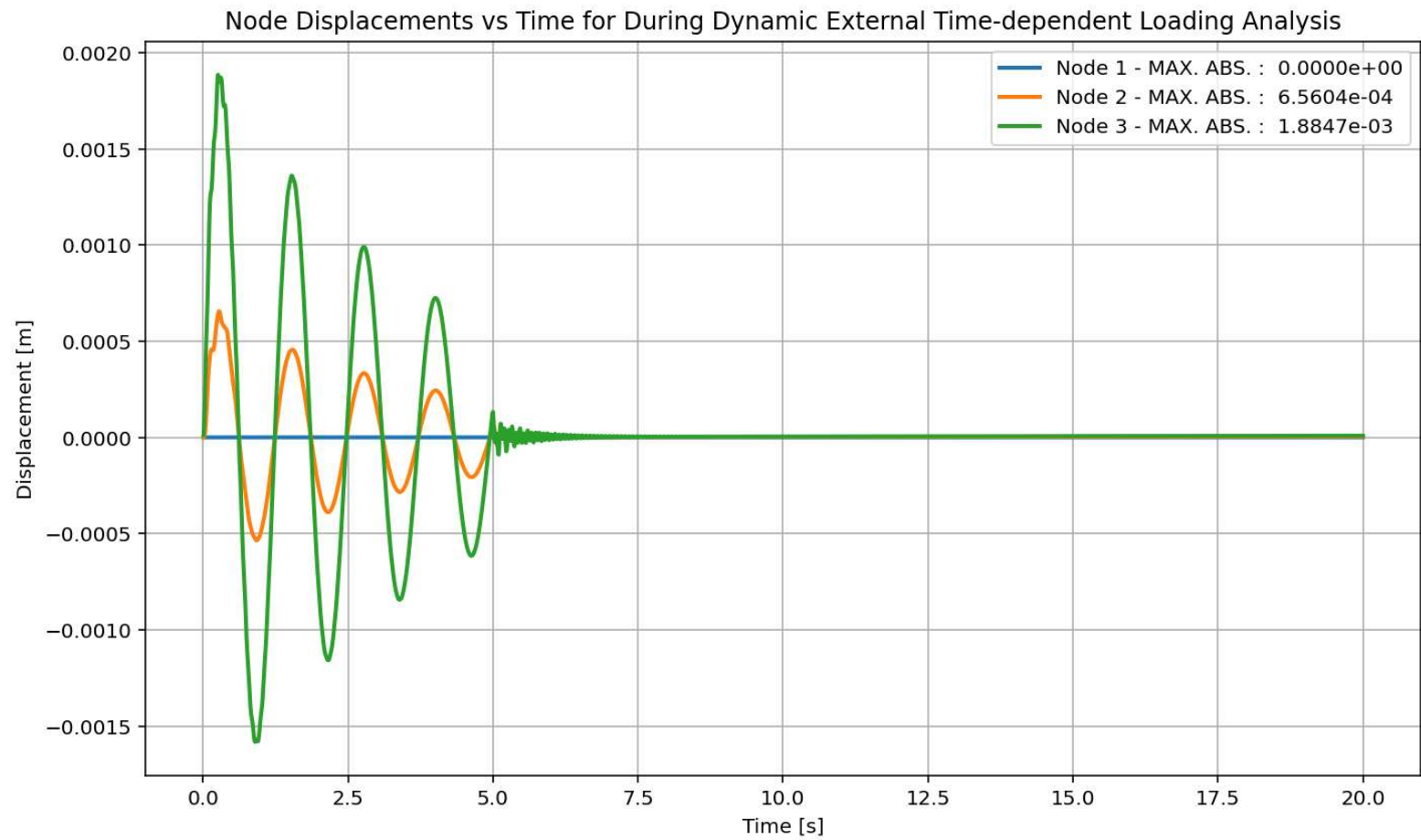
Velocity [m/s]

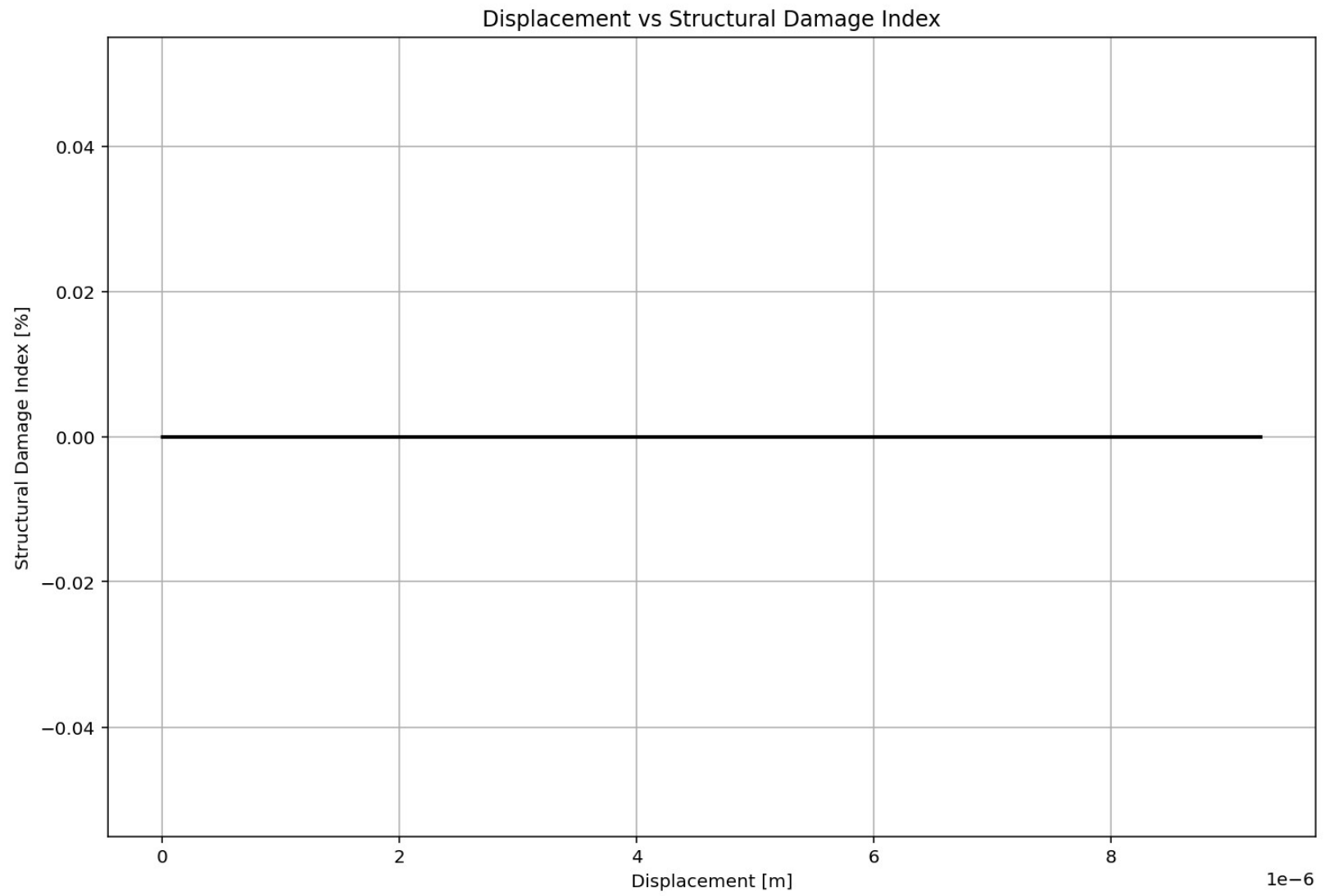
Acceleration [m/s²]

Period of Structure During Dynamic External Time-dependent Loading Analysis









FREE-VIBRATION ANALYSIS

Spyder (Python 3.12)

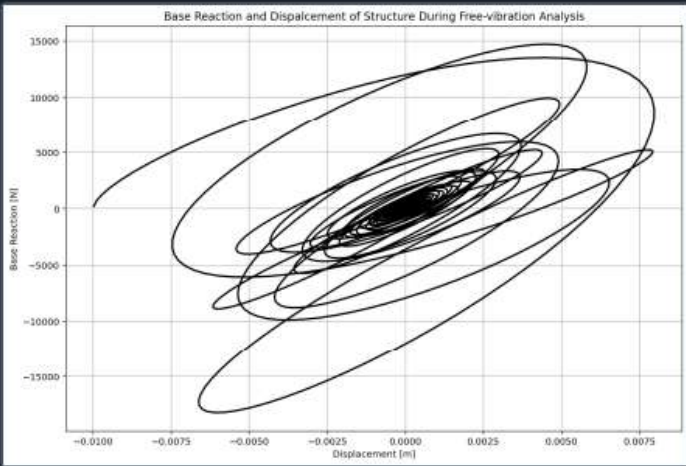
File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\De\l\Desktop\OPENSEES_FILES\+EIGHT_ANALY...TURE_MDOF_8_ANA\P(t)_SOIL_STRUCTURE_MDOF_8_ANA.py

P(t)_SOIL_STRUCTURE_MDOF_8_ANA.py X

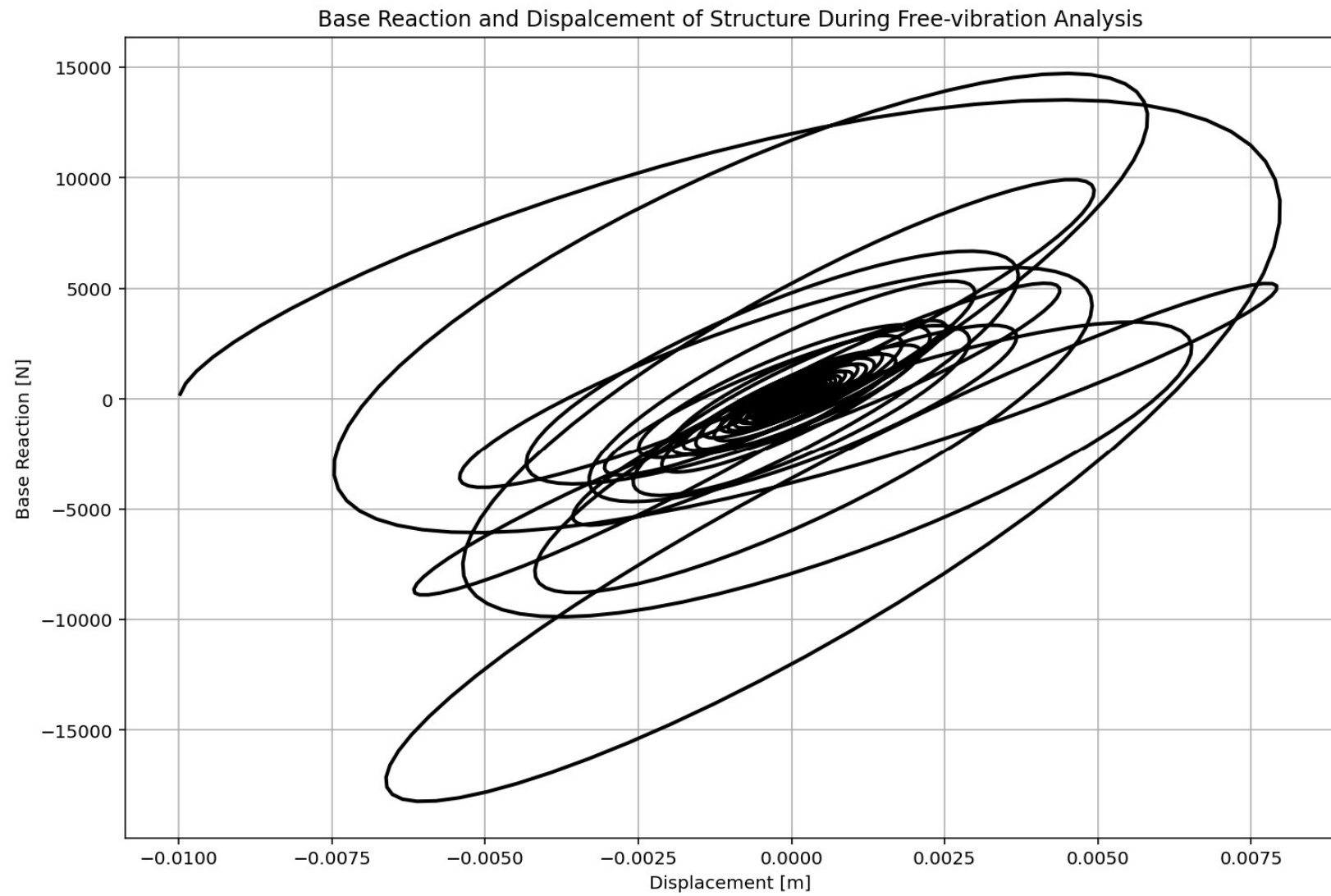
```
606 if ANAL TYPE == 'FREE-VIBRATION': # DYNAMIC TIME-HISTORY ANALYSIS
607     %% DEFINE PARAMETERS FOR FREE-VIBRATION ANALYSIS
608     u0 = -0.010 # [m] Initial displacement
609     v0 = 0.000015 # [m/s] Initial velocity
610     a0 = 0.000065 # [m/s^2] Initial acceleration
611     IU = True # Free Vibration with Initial Displacement
612     IV = True # Free Vibration with Initial Velocity
613     IA = True # Free Vibration with Initial Acceleration
614     duration = 20.0 # [s] Analysis duration
615     dt = 0.001 # [s] Time step
616     DR = 0.03 # Damping Ratio
617
618     if IU == True:
619         # Define initial displacment
620         ops.setNodeDisp(center_node, 1, u0, '-commit') # INFO LINK: https://openseespy
621     if IV == True:
622         # Define initial velocity
623         ops.setNodeVel(center_node, 1, v0, '-commit') # INFO LINK: https://openseespy
624     if IA == True:
625         # Define initial acceleration
626         ops.setNodeAccel(center_node, 1, a0, '-commit') # INFO LINK: https://openseespy
627
628     ops.constraints('Plain')
629     ops.numberer('Plain')
630     ops.system('BandGeneral')
631     ops.test('NormDispIncr', MAX_TOLERANCE, MAX_ITERATIONS) # INFO LINK: https://openseespy
632     #ops.integrator('CentralDifference') # JUST FOR LINEAR ANALYSIS - INFO LINK: http
633     alpha=0.5; beta=0.25;
634     ops.integrator('Newmark', alpha, beta) # INFO LINK: https://openseespydoc.readthedocs.io/en/latest
635     #alpha=2/3; gamma=1.5-alpha; gamma=1.5-alpha; beta=(2-alpha)**2/4;
636     #ops.integrator('HHT', alpha, gamma, beta) # INFO LINK: https://openseespydoc.readthedocs.io/en/latest
637     ops.algorithm('Newton') # INFO LINK: https://openseespydoc.readthedocs.io/en/latest
638     ops.analysis('Transient') # INFO LINK: https://openseespydoc.readthedocs.io/en/latest
639
```

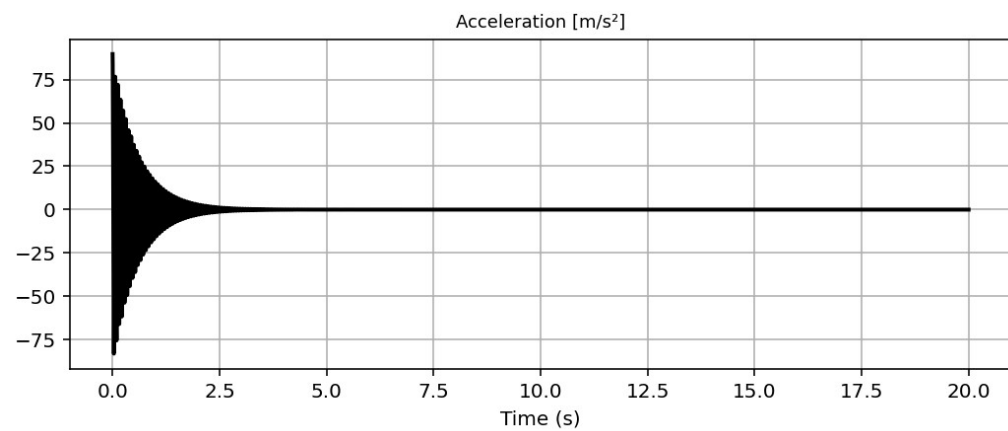
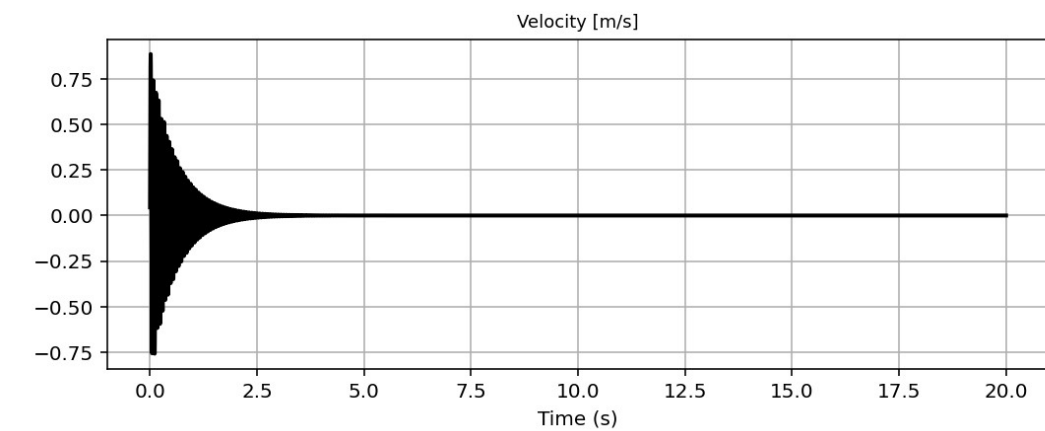
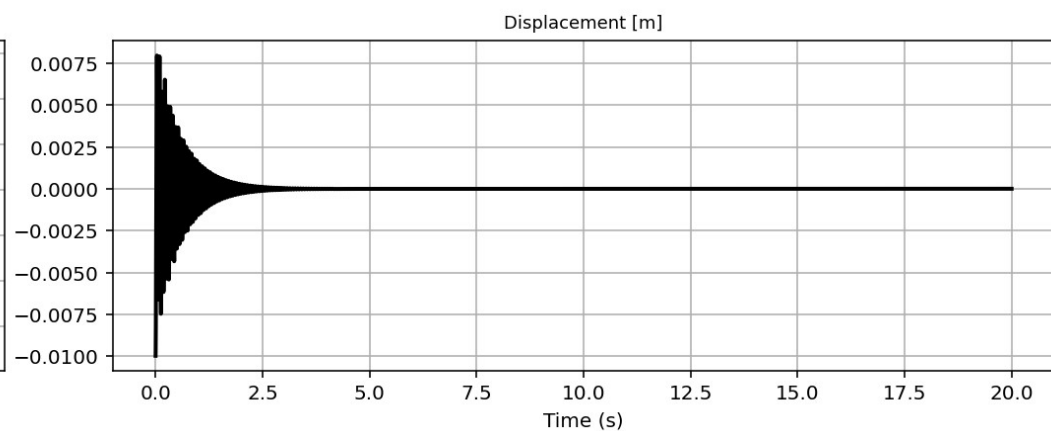
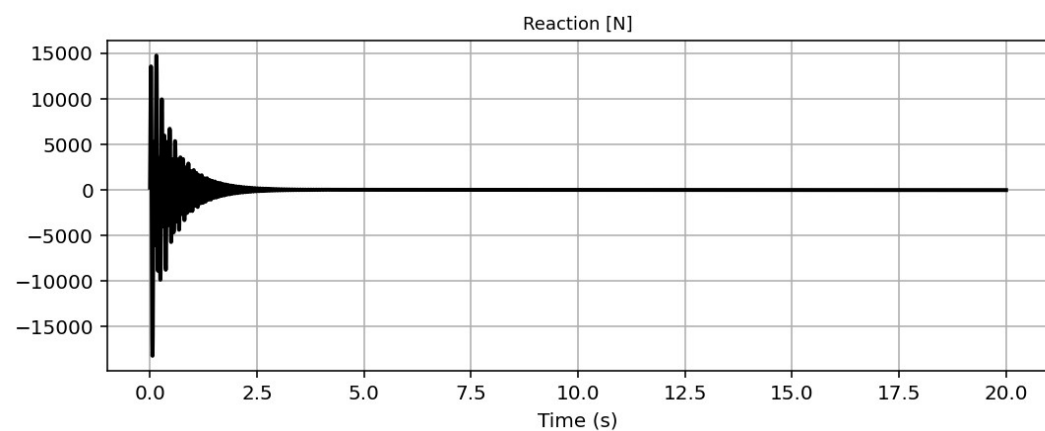
Base Reaction and Displacement of Structure During Free-vibration Analysis



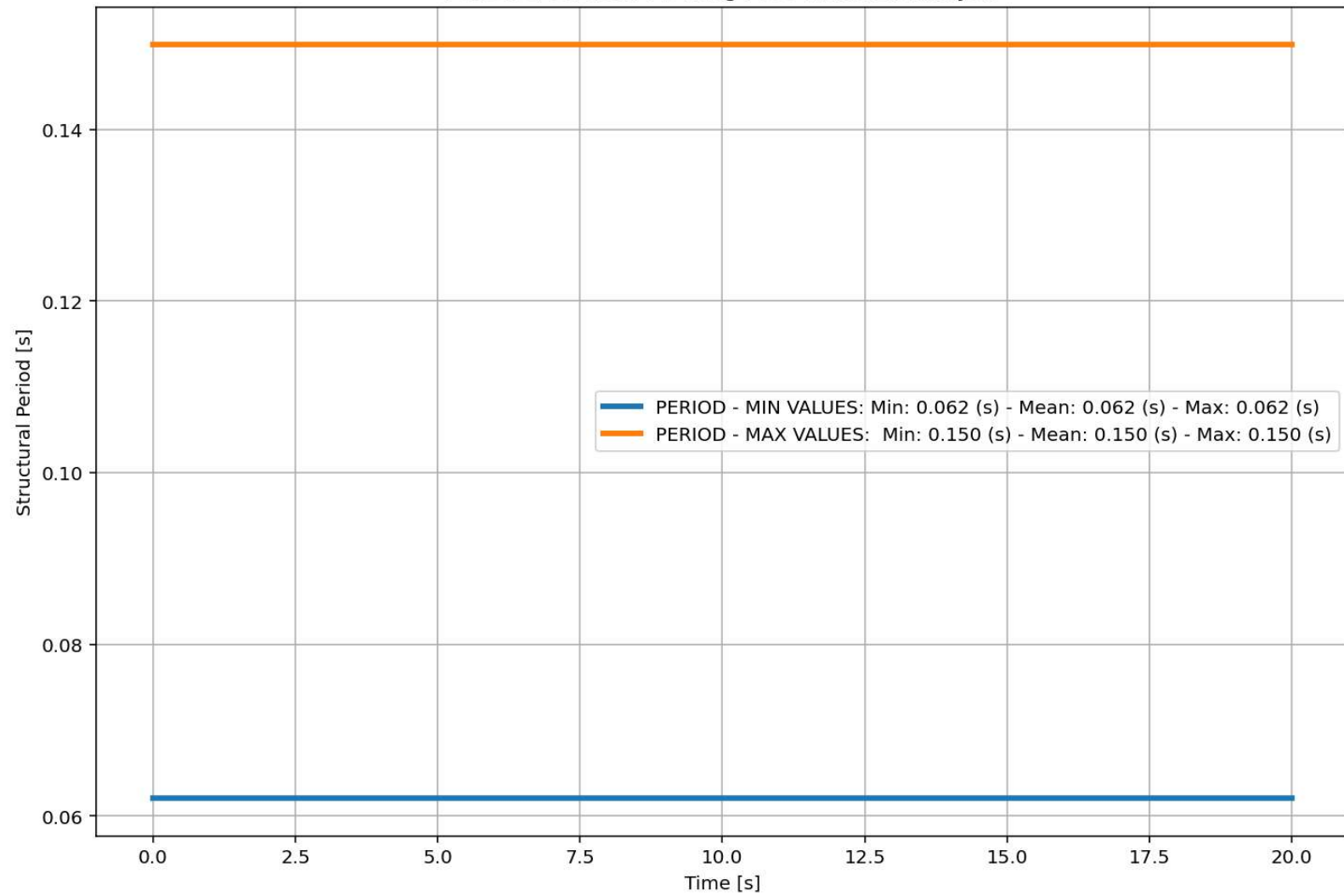
IPython Console Files Help Variable Explorer Debugger Plots History

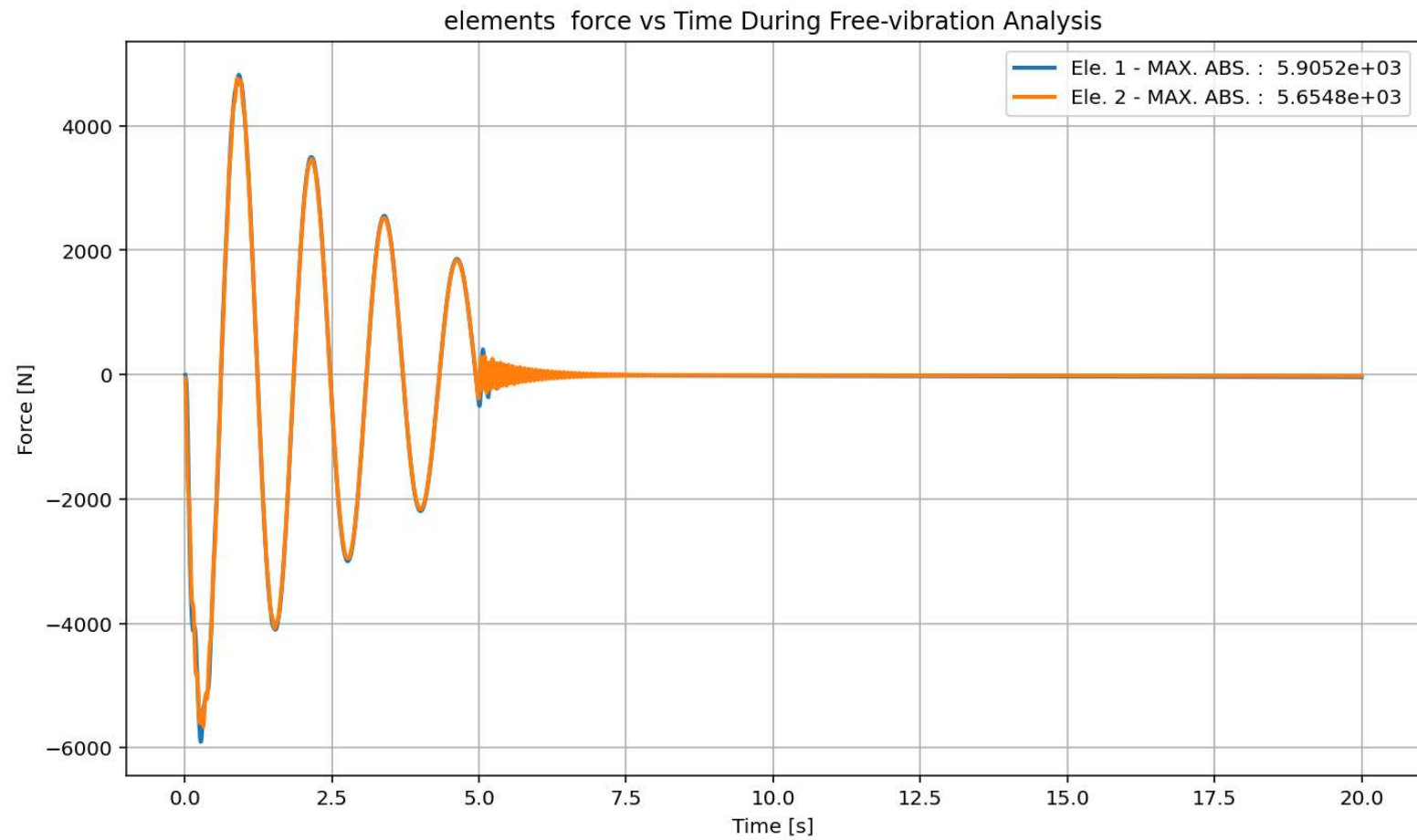
Inline Conda: anaconda3 (Python 3.12.7) ✓ LSP: Python Line 230, Col 55 UTF-8 CRLF RW Mem 47%



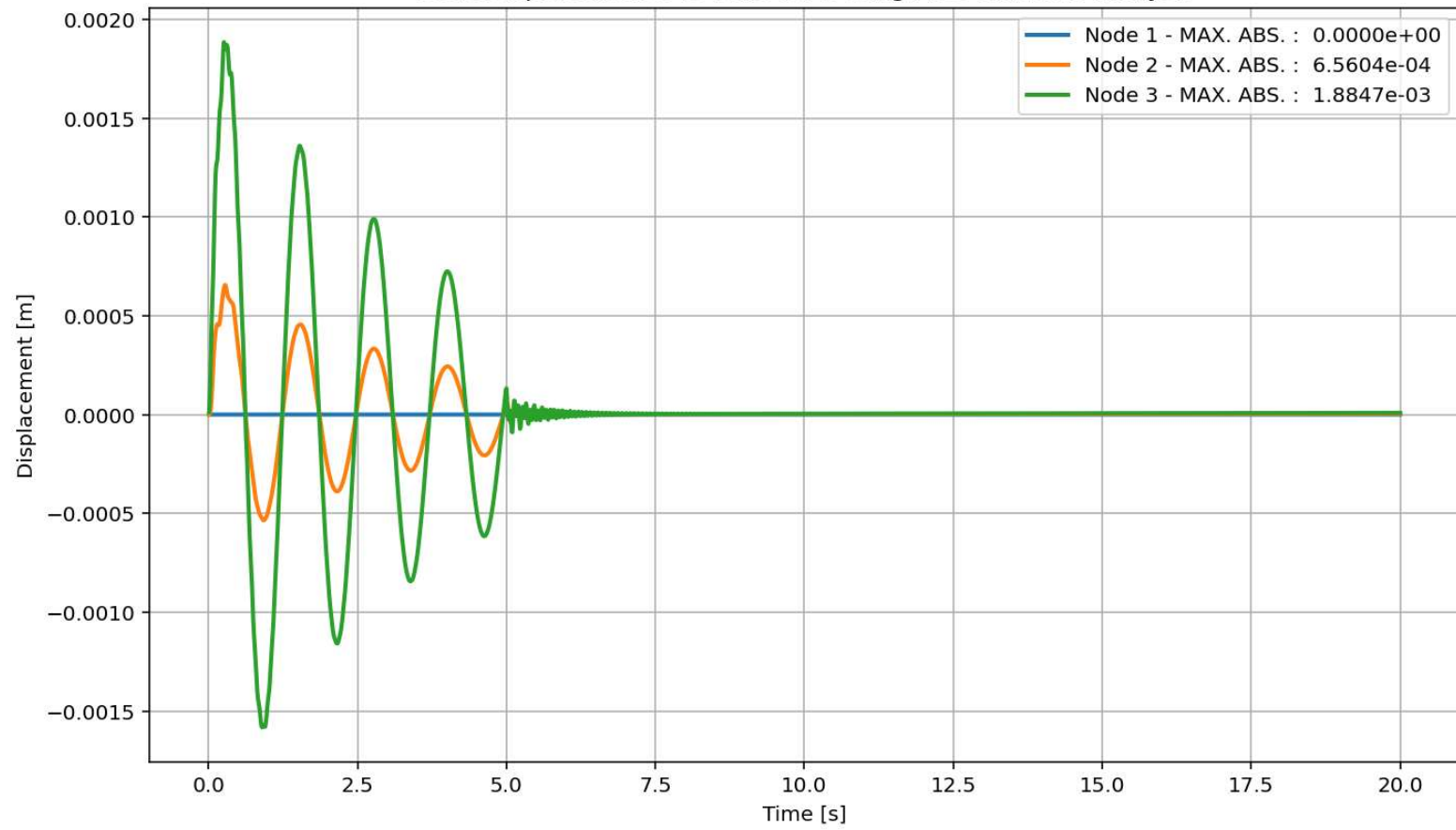


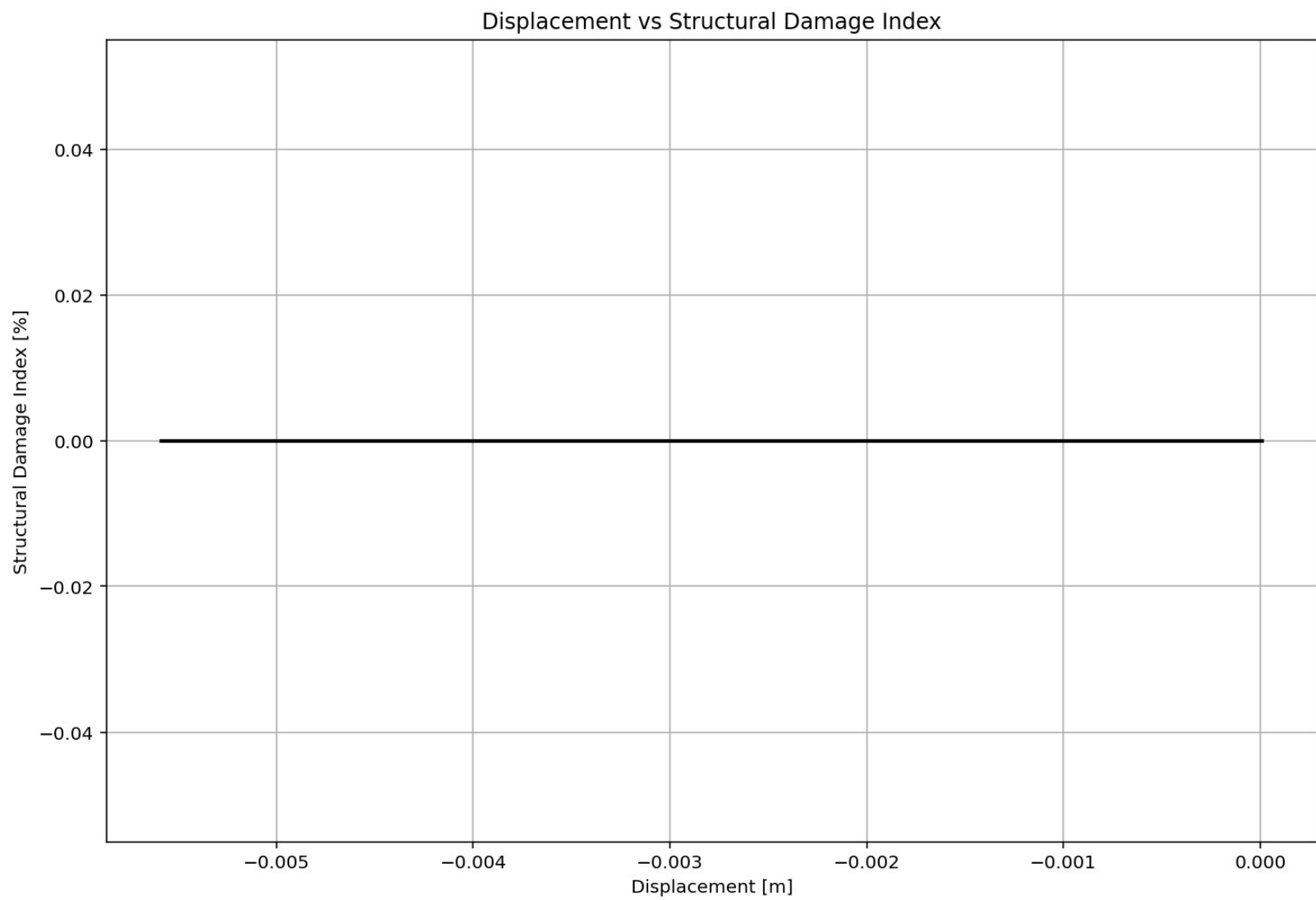
Period of Structure During Free-vibration Analysis

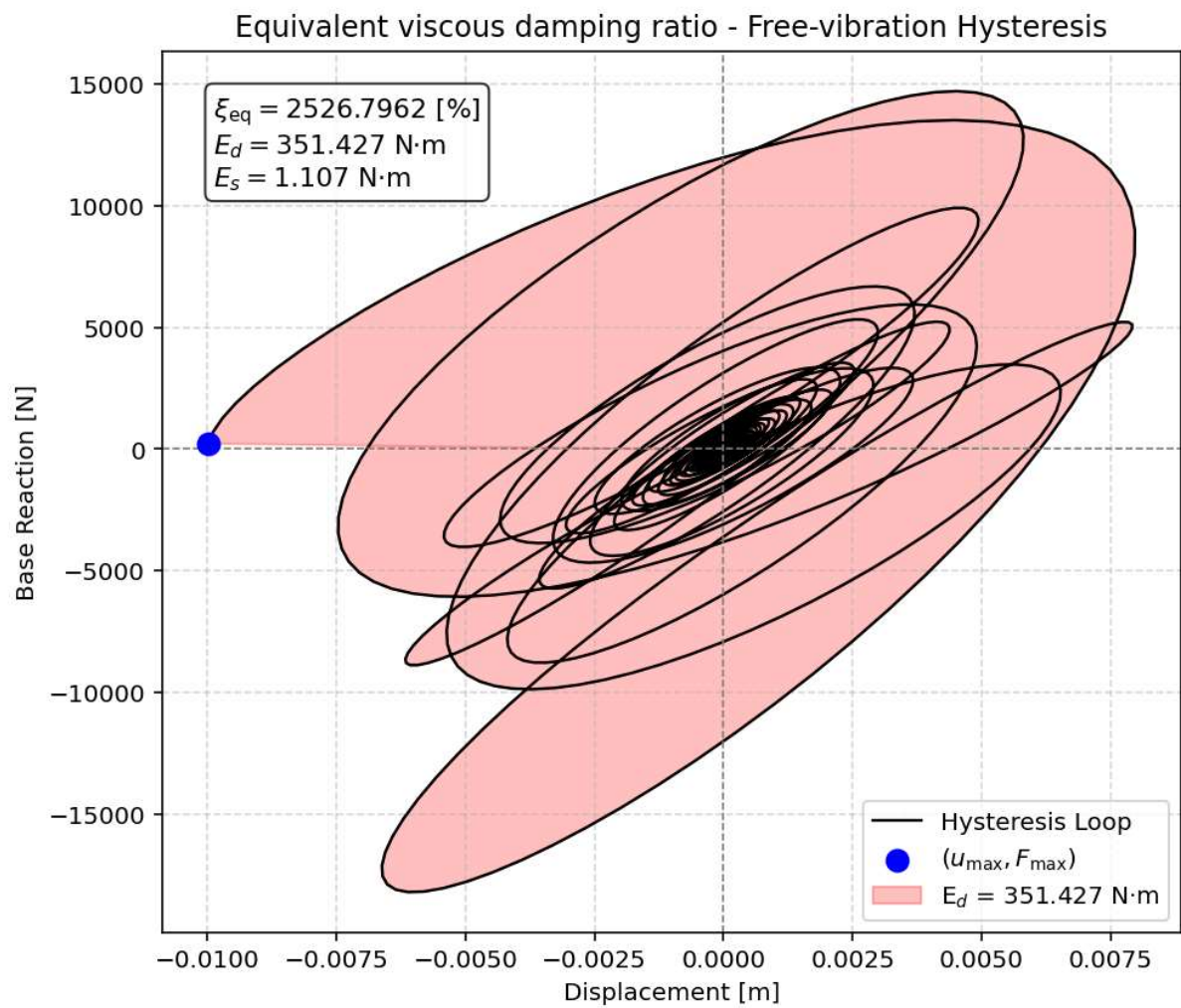




Node Displacements vs Time for During Free-vibration Analysis



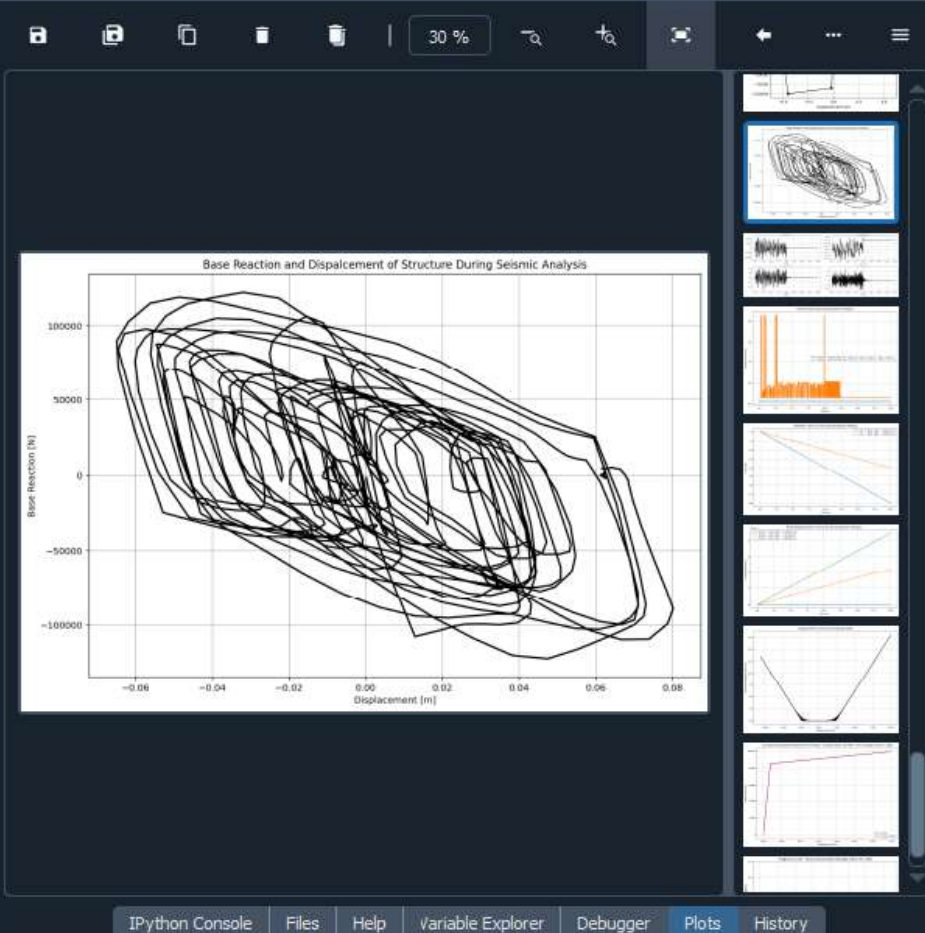




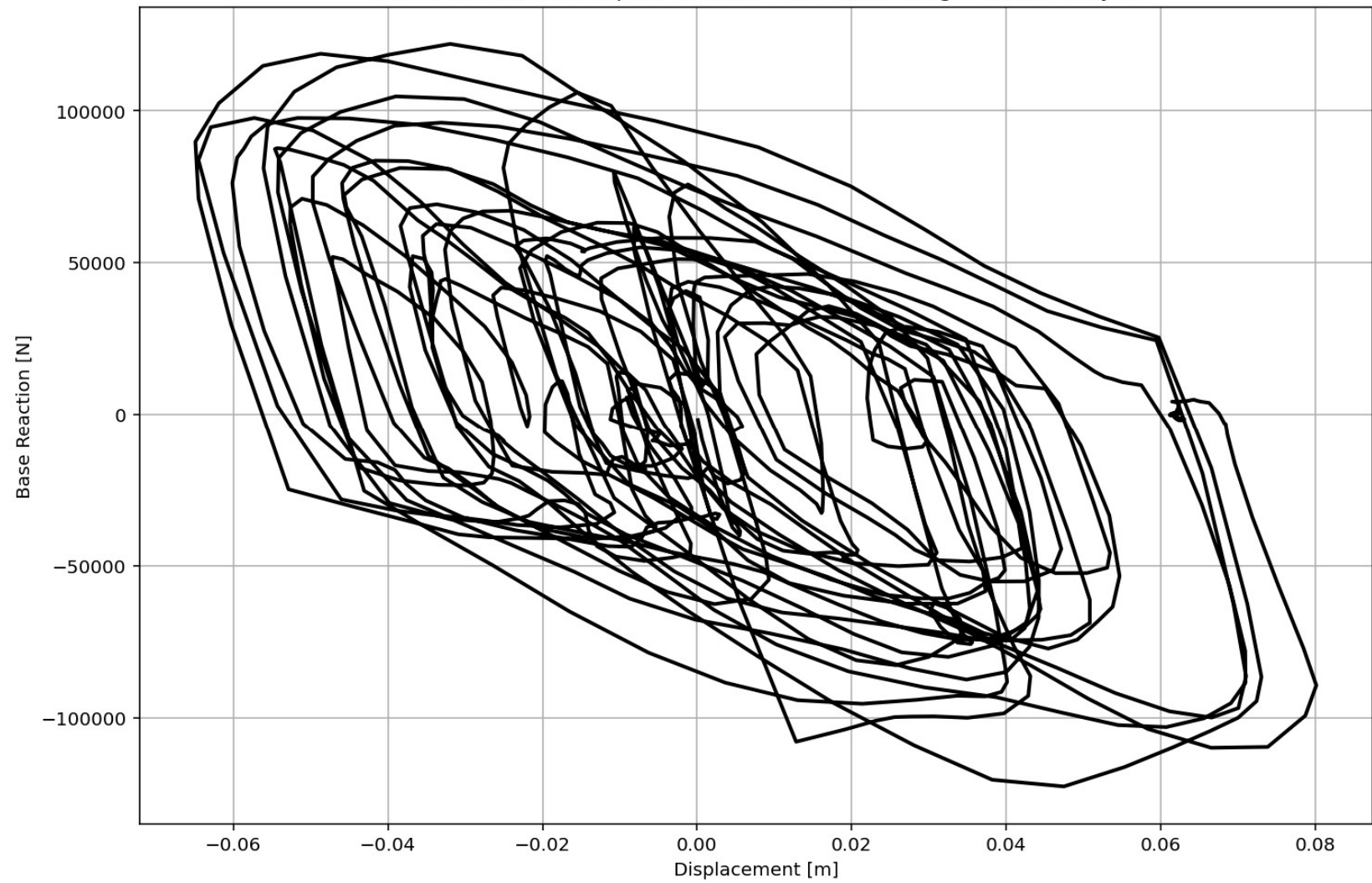
SEISMIC ANALYSIS


```

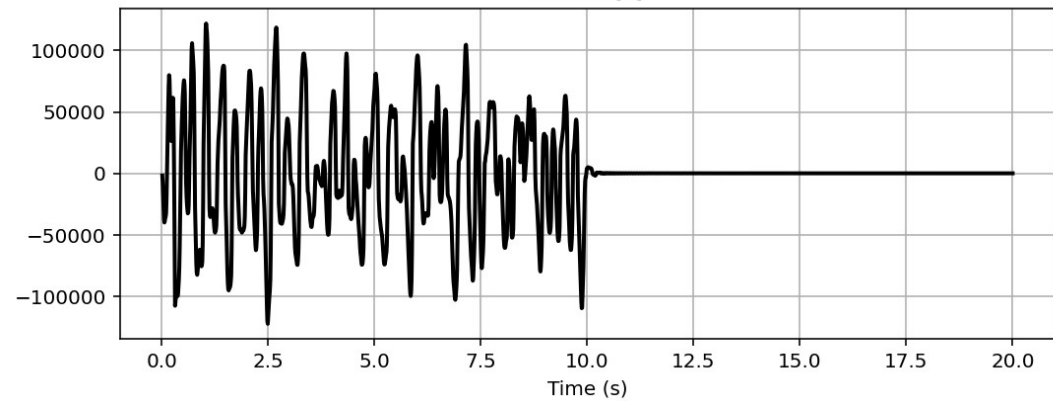
687 if ANAL_TYPE == 'SEISMIC': # DYNAMIC TIME-HISTORY ANALYSIS
688     %% DEFINE PARAMETERS FOR SEISMIC ANALYSIS
689     duration = 20.0           # [s] Analysis duration
690     dt = 0.01                # [s] Time step
691     GMfact = 9810             # [mm/s2] standard acceleration of gravity or st
692     SSF_X = 0.10              # Seismic Acceleration Scale Factor in X Direct
693     SSF_Y = 0.10              # Seismic Acceleration Scale Factor in Y Direct
694     iv0_X = 0.0005            # [mm/s] Initial velocity applied to the node
695     iv0_Y = 0.0005            # [mm/s] Initial velocity applied to the node
696     st_iv0 = 0.0              # [s] Initial velocity applied starting time
697     SEI = 'X'                 # Seismic Direction
698     DR = 0.03                 # Damping ratio
699
700 # Define time series for input motion (Acceleration time history)
701 if SEI == 'X':
702     SEISMIC_TAG_01 = 100
703     ops.timeSeries('Path', SEISMIC_TAG_01, '-dt', dt, '-filePath', 'Ground_Acceler
704     # Define load patterns
705     # pattern UniformExcitation $patternTag $dof -accel $tsTag <-vel0 $vel0> <-fac
706     ops.pattern('UniformExcitation', SEISMIC_TAG_01, 1, '-accel', SEISMIC_TAG_01,
707 if SEI == 'Y':
708     SEISMIC_TAG_02 = 200
709     ops.timeSeries('Path', SEISMIC_TAG_02, '-dt', dt, '-filePath', 'Ground_Acceler
710     ops.pattern('UniformExcitation', SEISMIC_TAG_02, 2, '-accel', SEISMIC_TAG_02,
711 if SEI == 'XY':
712     SEISMIC_TAG_01 = 100
713     ops.timeSeries('Path', SEISMIC_TAG_01, '-dt', dt, '-filePath', 'Ground_Acceler
714     # Define load patterns
715     # pattern UniformExcitation $patternTag $dof -accel $tsTag <-vel0 $vel0> <-fac
716     ops.pattern('UniformExcitation', SEISMIC_TAG_01, 1, '-accel', SEISMIC_TAG_01,
717     SEISMIC_TAG_02 = 200
718     ops.timeSeries('Path', SEISMIC_TAG_02, '-dt', dt, '-filePath', 'Ground_Acceler
719     ops.pattern('UniformExcitation', SEISMIC_TAG_02, 2, '-accel', SEISMIC_TAG_02,
720     print('Seismic Defined Done.')
    
```



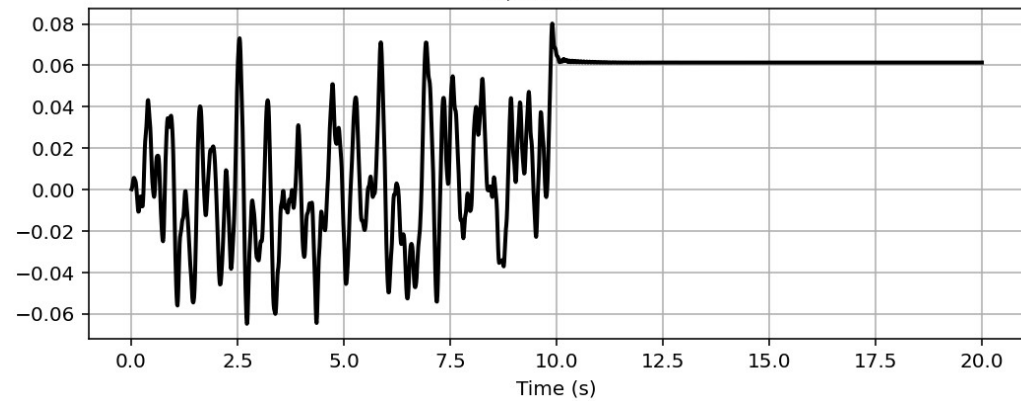
Base Reaction and Dispalcement of Structure During Seismic Analysis



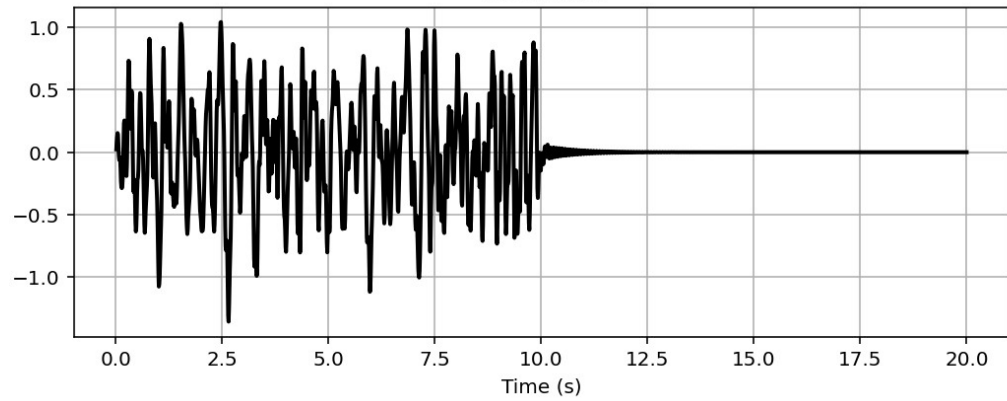
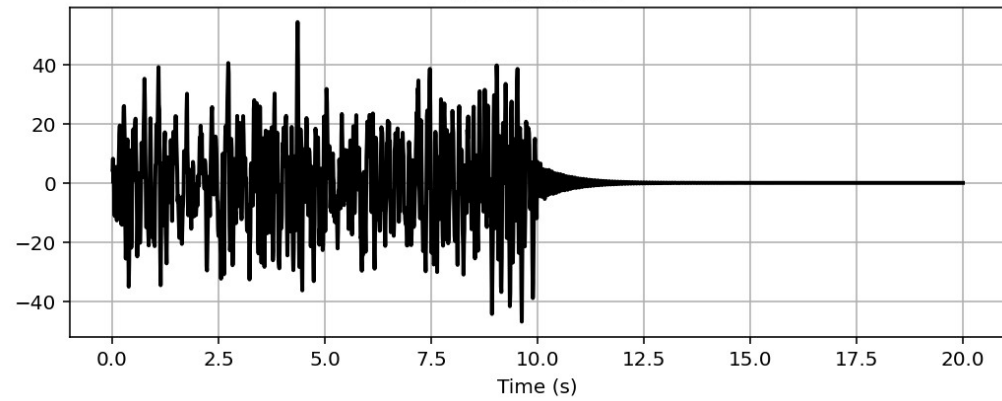
Reaction [N]



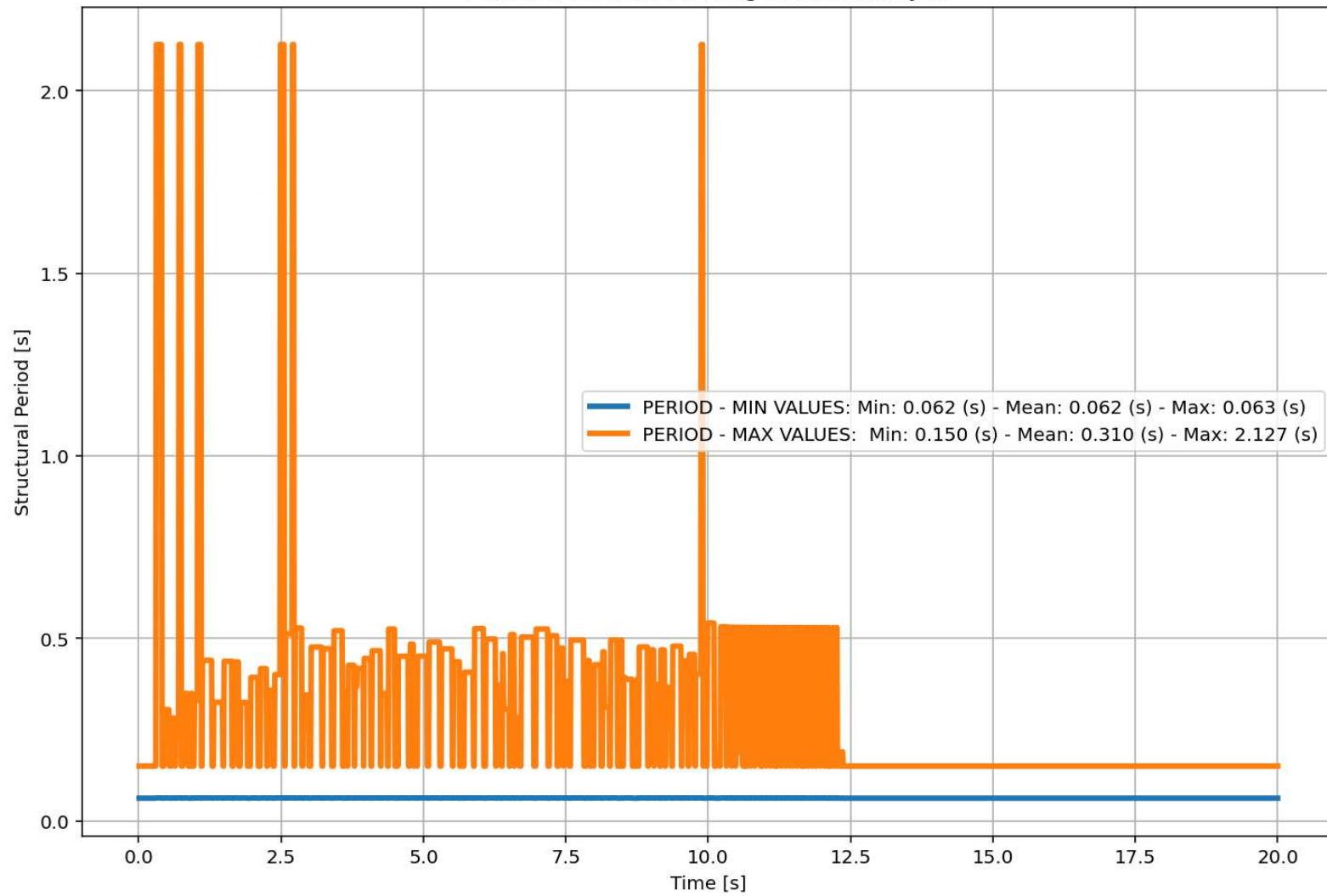
Displacement [m]

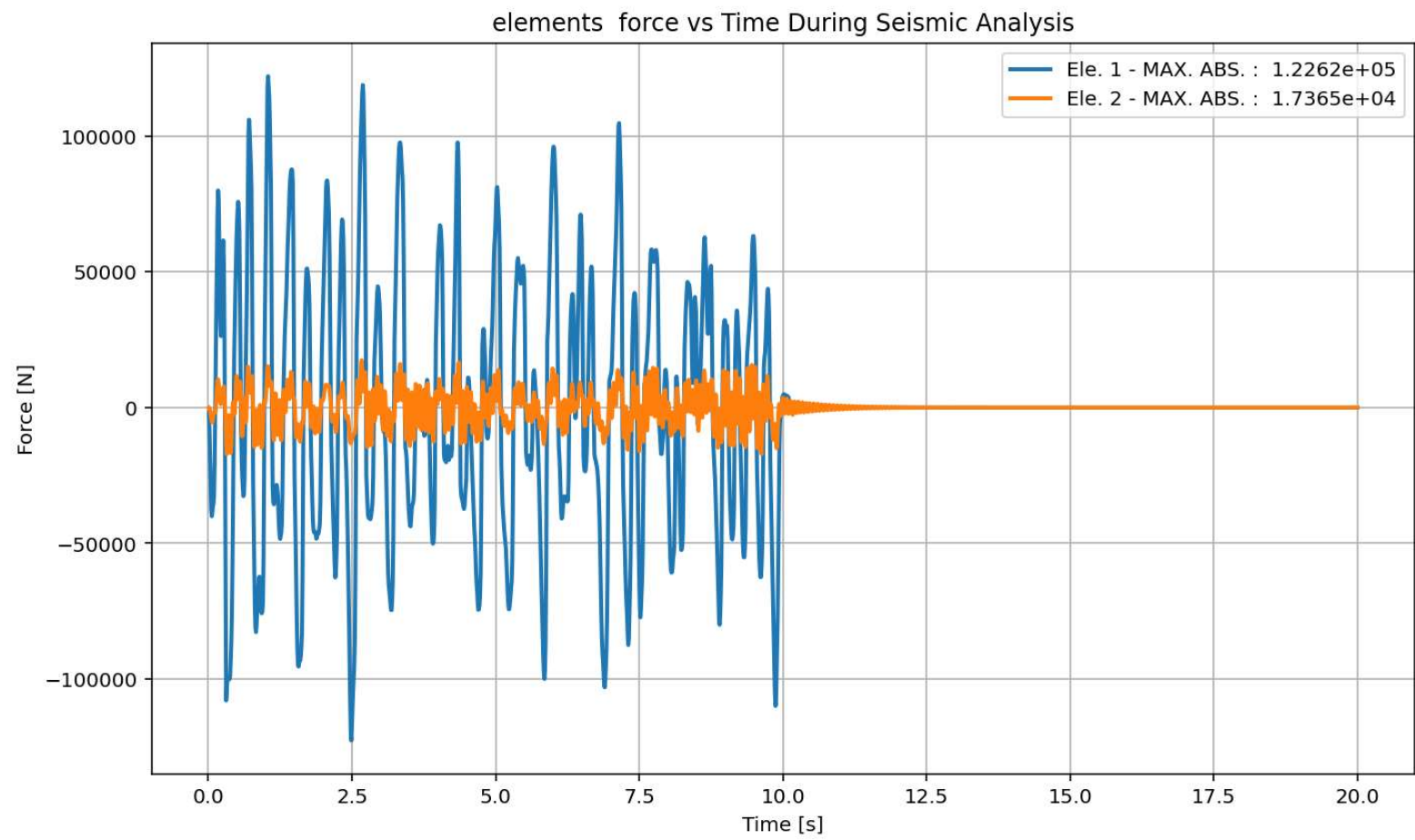


Velocity [m/s]

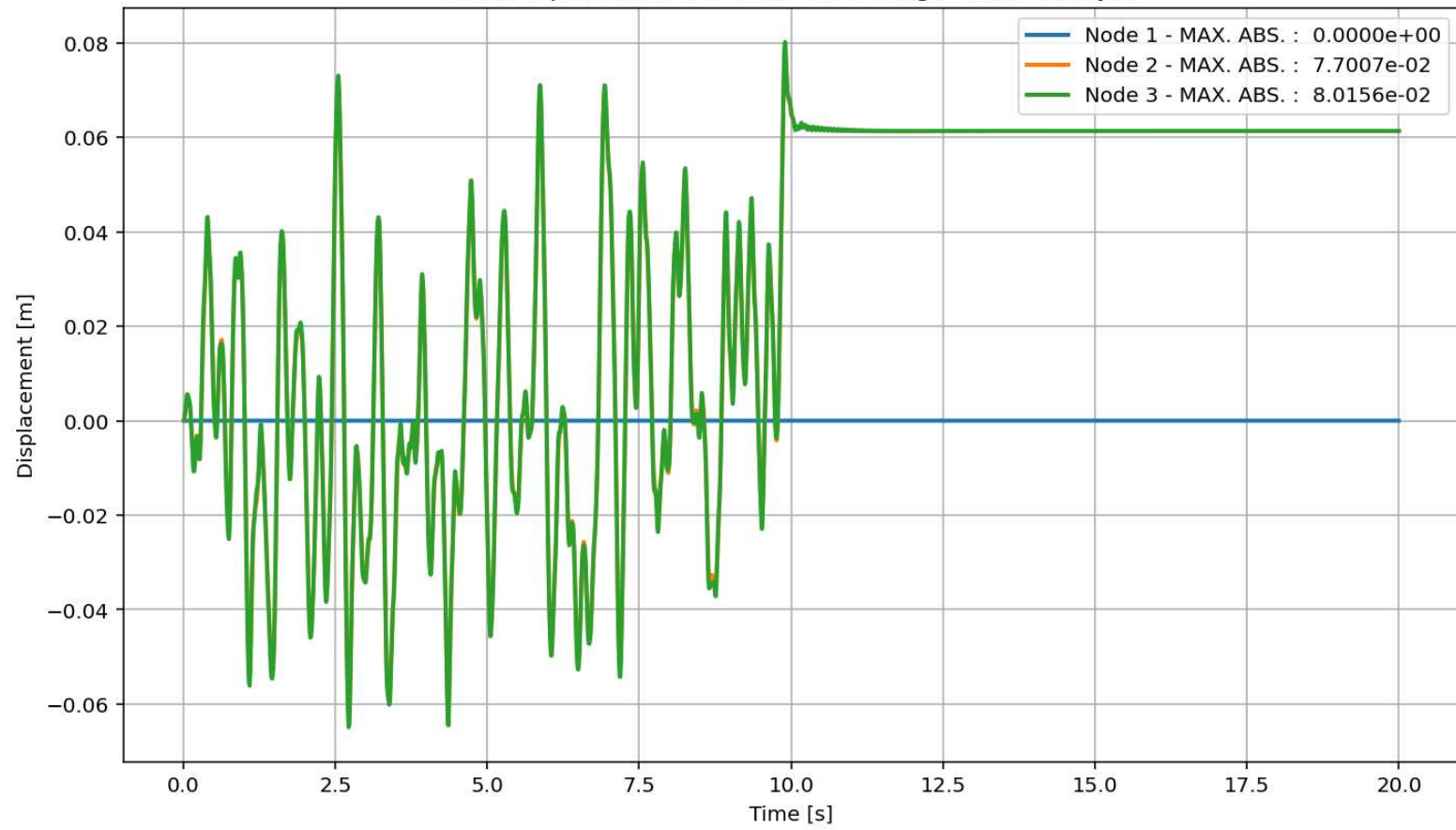
Acceleration [m/s²]

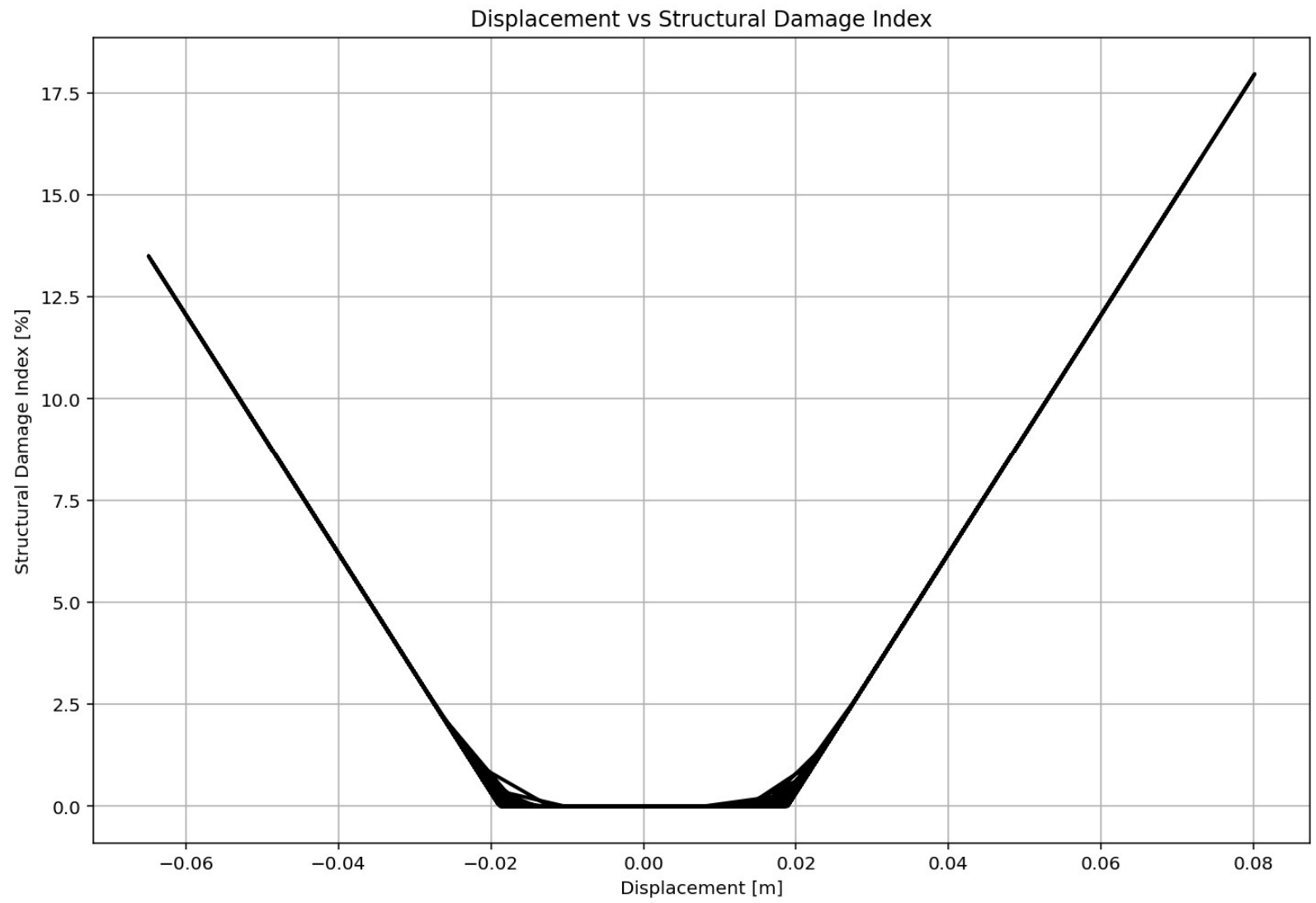
Period of Structure During Seismic Analysis



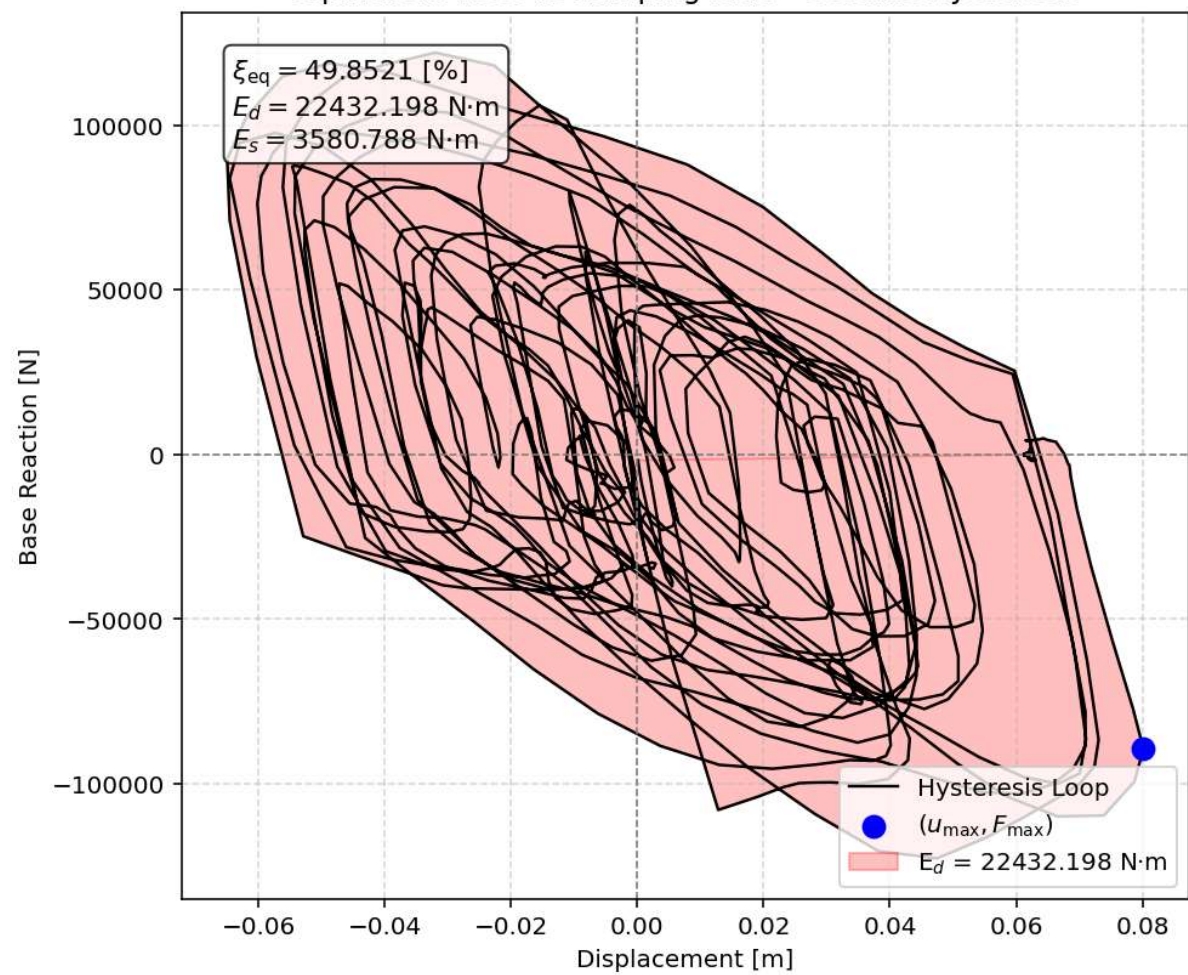


Node Displacements vs Time for During Seismic Analysis

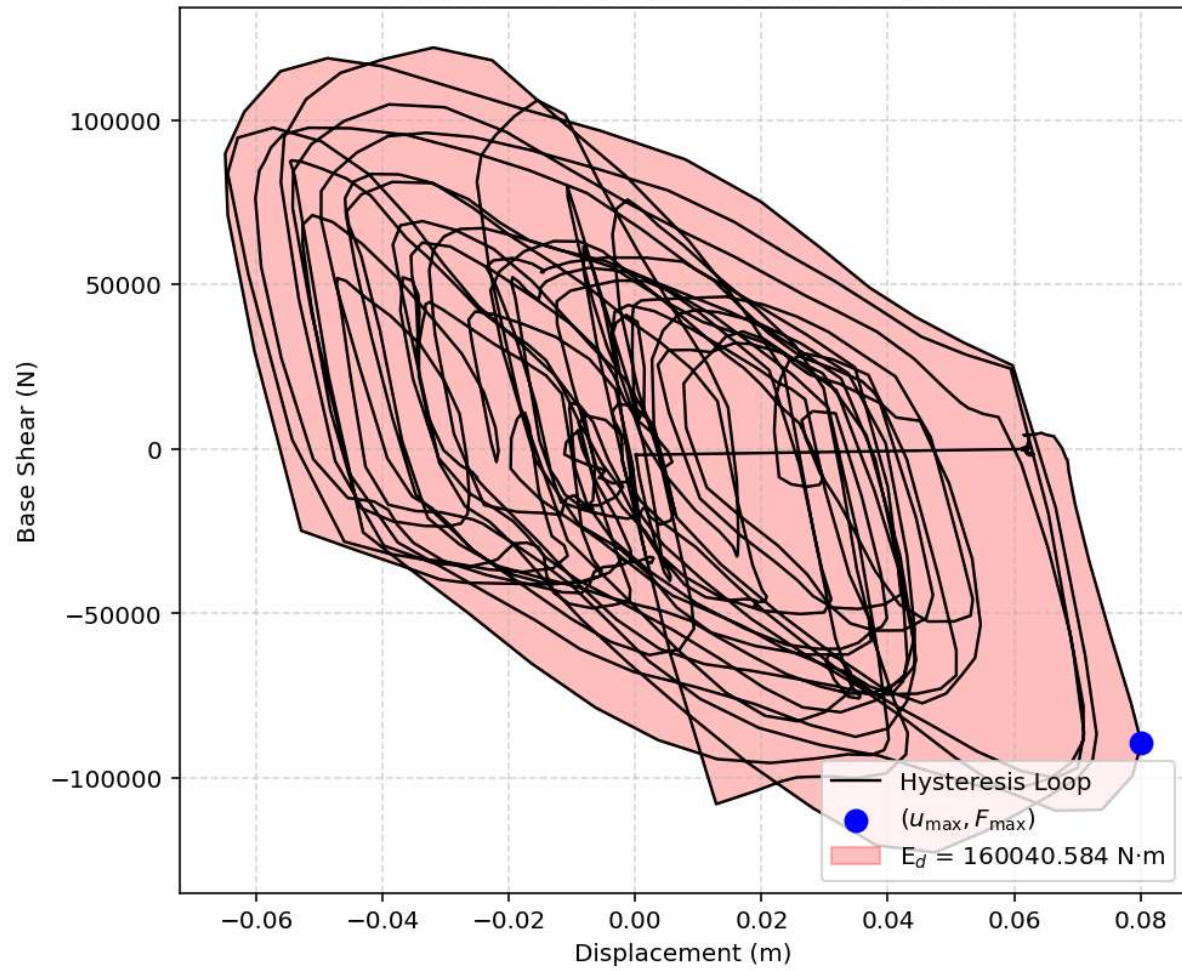


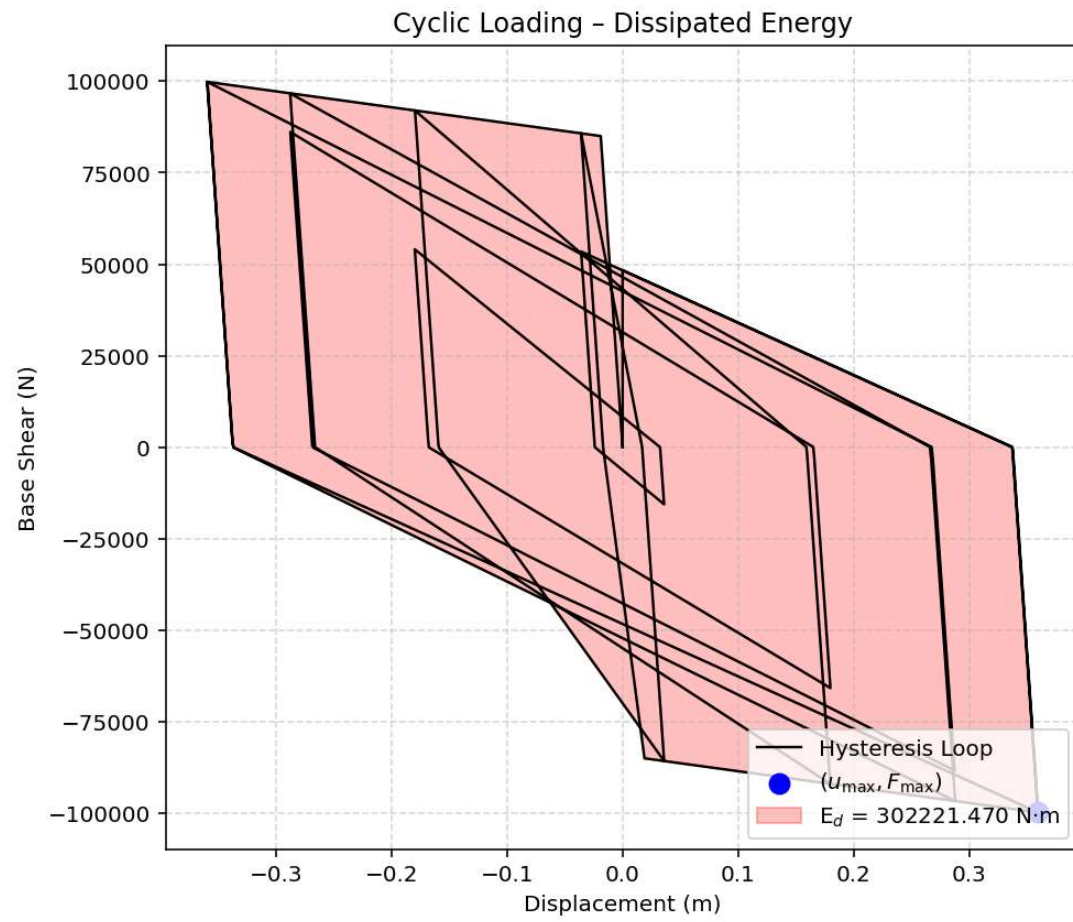


Equivalent viscous damping ratio - Seismic Hysteresis



Earthquake Response - Dissipated Energy





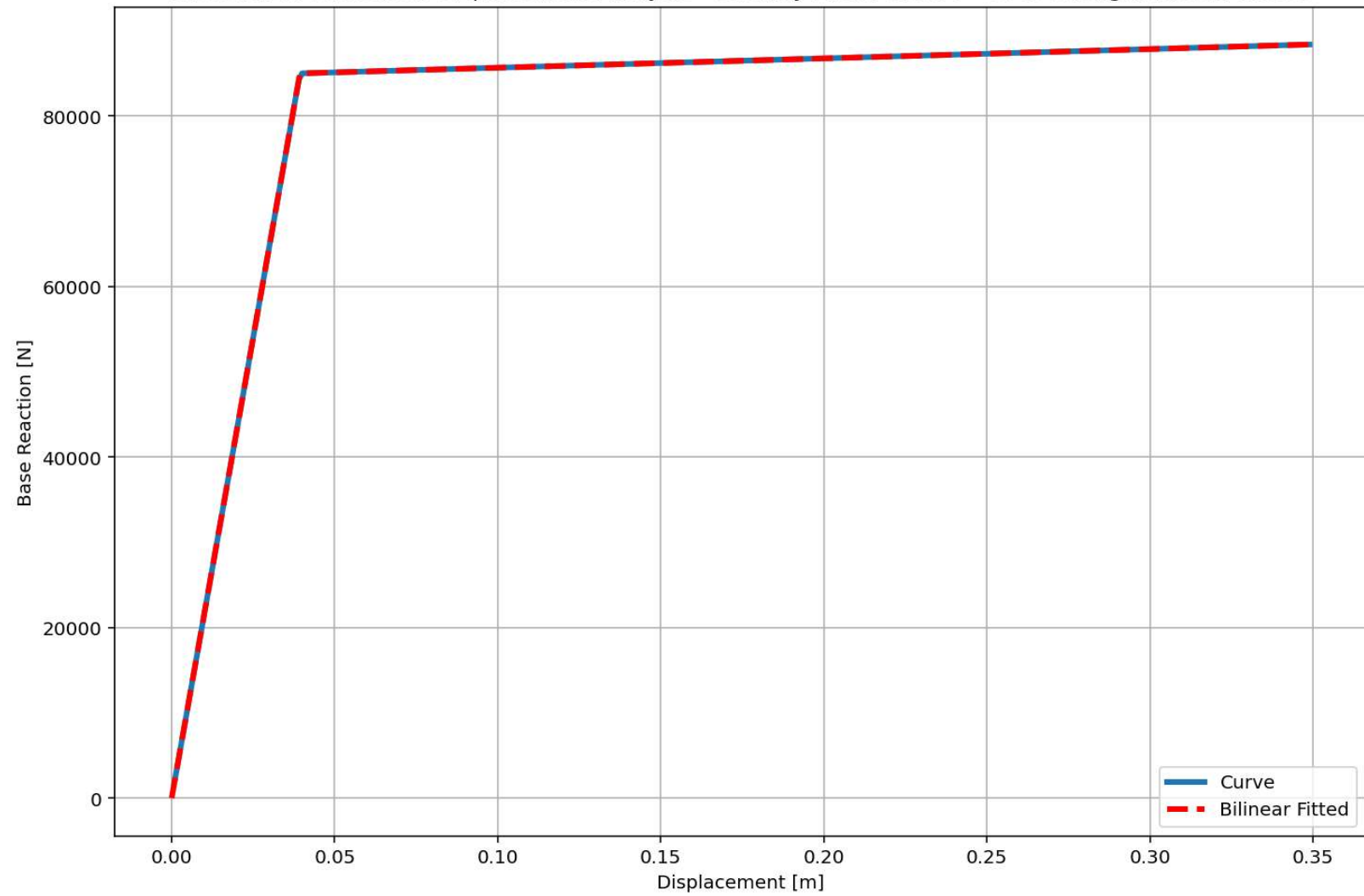
Dissipated Energy from Earthquake= 160040.58 N·m

Dissipated Energy from Cyclic Displacement= 302221.47 N·m

DISSIPATED ENERGY CAPACITY INDEX: 52.955 [%]

ZONE 6: SEVERE-LOW DAMAGE

Last Data of BaseShear-Displacement Analysis - Ductility Ratio: 8.8696 - Over Strength Factor: 1.0399



+=====+

= Analysis curve fitted =

Disp Base Shear

[[0.00000000e+00 0.00000000e+00]

[1.88855309e-02 8.49848890e+04]

[3.49000000e-01 9.93381699e+04]]

+=====+

+-----+

Structure Elastic Stiffness : 4500000.00

Structure Plastic Stiffness : 284636.59

Structure Tangent Stiffness : 43479.71

Structure Ductility Ratio : 18.48

Structure Over Strength Factor: 1.17

+-----+

Over Strength Coefficient (Ω): 1.1745

Displacement Ductility Ratio (μ): 19.0621

Ductility Coefficient ($R\mu$): 19.0621

Structural Behavior Coefficient (R): 22.3886

Structural Ductility Damage Index in X Direction: 17.9620 (%)

Seismic Fragility Curves by Damage State from DYN. TIME-HISTORY ANA. DUCTILITY DAMAGE INDEX [%]

